

Particle Flow Filter and Differentiable Particle Filter

Haowu Duan

April 10, 2026

Contents

1	The big picture	3
1.1	State-Space Modeling in Financial Markets	3
1.1.1	Filtering	3
1.1.2	Bayesian Inference	3
1.2	Document Roadmap	4
2	Kalman Filter	5
2.1	Overview	5
2.2	Algorithm	5
2.3	Numerical stability analysis for Kalman filter	7
2.4	Numerical experiments and failure on nonlinear Models	8
3	EKF and UKF	9
3.1	EKF	9
3.2	UKF	10
3.3	EKF/UKF on non-linear models	12
4	Particle-Based Methods	13
4.1	Importance Sampling and Sequential Monte Carlo	13
4.2	Bootstrap Particle Filter	14
4.2.1	Algorithm	14
4.2.2	Particle Degeneracy and Resampling	14
4.3	EDH/LEDH Flow Filter	15
4.3.1	Use Fokker-Planck to reach the target distribution	15
4.3.2	Determine the evolution kernel	16
4.3.3	EDH vs. LEDH: incremental improvements in linearization quality.	16
4.3.4	Algorithm	17
4.3.5	Experiments	18
4.4	Invertible Flow Filter	18
4.4.1	Why do we need Jacobian computation?	18
4.5	Stochastic Flow Filter	21
4.6	Kernel Flow Filter	23
4.6.1	Overview	23
4.6.2	Derivation	23
4.6.3	Matrix Kernel Preventing Collapse	25
4.6.4	Algorithm	25
4.7	Filter Comparison Summary	25
5	Resampling and Optimal Transport	26
5.1	Two Problems with Resampling	26
5.2	Soft Resampling	26
5.3	Entropy-Regularized OT (Sinkhorn) Resampling	27
5.4	The Derivative Chain through Resampling	28
5.5	Approximate vs Implicit Sinkhorn Backward	30
5.5.1	The Extrapolation Approximation	30

5.5.2	Implicit Differentiation at the Fixed Point	30
5.6	Neural Acceleration	31
6	Parameter Estimation with Differentiable Particle Filters	33
6.1	Markov Chain Monte Carlo	33
6.1.1	Metropolis-Hastings	33
6.1.2	Hamiltonian Monte Carlo	33
6.2	MAP as a Pipeline Smoke Test	35
6.3	Differentiable Resampling and the Gradient Chain	35
6.4	Benchmark: Linear Gaussian Model	35
6.4.1	MCMC Diagnostics	35
6.4.2	1-D Model, 1 Parameter	36
6.4.3	Multi-Dimensional Model, 1 Parameter	36

1 The big picture

1.1 State-Space Modeling in Financial Markets

Financial systems are driven by latent factors that shape observable market data. A canonical example is stochastic volatility: the variance of asset returns is not constant but evolves as a hidden process, so what we observe (returns) is a noisy function of a latent state (log-variance) that we cannot directly measure. Similar structure appears in limit order book dynamics and macroeconomic regime shifts. These problems are naturally cast as continuous state-space models (SSMs), which is the focus of this document.

The setup is simple. We observe a time series $\{z_k\}$, $k = 1, 2, \dots, T$. The real driver is a hidden state $\{x_k\}$ that we cannot observe directly. The most general SSM writes:

$$\begin{aligned} x_k &= f(x_{k-1}, w_k) \\ z_k &= h(x_k, u_k) \end{aligned} \tag{1}$$

where w_k and u_k are mutually and temporally uncorrelated noises, f governs state evolution, and h maps states to observations. We only ever see z_k . The state x_k may have the same dimension as z_k or higher — the latter case (partial observation) makes inference harder because information is lost in the mapping h .

A model has two components: a functional form and its parameters. In practice both are unknown. What we do is make an ansatz about the functional form, fit the parameters to data, and treat that ansatz as our best current understanding until a better one comes along. The observations themselves are agnostic to the model choice.

To develop and test numerical methods rigorously, we work in controlled experiments: fix a model and its parameters, generate synthetic data, and check whether our algorithms recover the known ground truth. This is the scope of this project. The two core tasks are **filtering** — estimating the hidden state x_k from observations — and **Bayesian inference** — estimating the model parameters θ from the same observations.

1.1.1 Filtering

Having a good model is only half the battle; we also need a reliable implementation. In research, novel filtering algorithms are developed and evaluated on synthetic data where the ground truth is known, allowing clear performance measurement. Given the state evolution f and observation model h , the goal is to estimate the hidden state x_k via the filtering distribution $p(x_k | z_{1:k})$. This is obtained recursively:

$$\begin{aligned} \textbf{Prediction:} \quad p(x_k | z_{1:k-1}) &= \int p(x_k | x_{k-1}) p(x_{k-1} | z_{1:k-1}) dx_{k-1} \\ \textbf{Update:} \quad p(x_k | z_{1:k}) &= \frac{p(z_k | x_k) p(x_k | z_{1:k-1})}{p(z_k | z_{1:k-1})} \end{aligned} \tag{2}$$

In financial applications (and most real-world settings), we do not aim to recover an exact deterministic value of x_k . Instead, the useful quantities are typically the mean m_k and covariance P_k of the filtering distribution. The filtering problem is generally a high-dimensional path integral and is intractable except in special cases (e.g., linear Gaussian SSMs). Different filtering methods (EKF, UKF, particle filters, etc.) use distinct approximations and mathematical frameworks. The key requirement for any successful filtering algorithm is that it produces numerically stable and accurate approximations to $p(x_k | z_{1:k})$ for known model dynamics.

1.1.2 Bayesian Inference

Once we have a reliable filter, we can use it to estimate the model parameters θ . We keep the functional forms of f and h fixed and treat their parameters as unknowns to be learned. A good θ should make the observed sequence probable. This means maximizing the marginal likelihood $p_\theta(z_{1:T})$, or equivalently minimizing the negative log-likelihood:

$$L_{\text{MLE}}(\theta) = - \sum_{k=1}^T \log p_\theta(z_k | z_{1:k-1}). \tag{3}$$

This is MLE. In a Bayesian setting we also have a prior $p(\theta)$ encoding any hard knowledge about the parameters — for instance, a variance must be positive, or a correlation must lie in $(-1, 1)$. We then minimize:

$$L_{\text{MAP}}(\theta) = - \sum_{k=1}^T \log p_{\theta}(z_k \mid z_{1:k-1}) - \log p(\theta), \quad (4)$$

which gives MAP estimation. MAP is strictly more informative than MLE: the prior rules out parameter regions that are physically unreasonable or numerically dangerous, so the optimizer cannot wander into them.

1.2 Document Roadmap

We cover the two tasks in sequence. First, filtering. We start with the Kalman filter on linear Gaussian models — the only case where the prediction and update integrals are analytically tractable — and use it as a baseline throughout. We then cover the nonlinear extensions: EKF and UKF, which linearize or approximate the model to stay within the Gaussian framework. Next, particle-based methods, which abandon the Gaussian assumption entirely and represent the filtering distribution as a weighted set of samples. Within that family we discuss the Bootstrap Particle Filter (BPF), the EDH and LEDH flow filters (which move particles via a continuous transport rather than reweighting them), and their invertible variants. Second, Bayesian inference. Once the filter is differentiable, we can backpropagate through it to learn θ . We cover MAP estimation as a diagnostic baseline and HMC for full posterior sampling. The central challenge is making the resampling step differentiable so that gradients can flow back to θ . We discuss stop-gradient systematic resampling and soft resampling as reference points, but the main target is OT resampling and its neural-accelerated variant, which provides the richest gradient signal and is the primary method we will develop.

2 Kalman Filter

2.1 Overview

Linear Gaussian models are the easiest state space models. Take the following 1-d dimensional model for example,

$$\begin{aligned} x_k &= f x_{k-1} + w_{k-1}, & w_{k-1} &\sim \mathcal{N}(0, Q) \\ z_k &= h x_k + v_k, & v_k &\sim \mathcal{N}(0, R) \end{aligned} \quad (5)$$

where f and h are constant. Of course to ensure the stability of the sequence, both of them has to be smaller or equal to one. This problem can be exactly solved as we will later demonstrate that the distribution for x_k and z_k will stay as Gaussian through out the process and all quantities are analytically computable. The corresponding algorithm that is derived for linear gaussian models is the Kalman Filter (KF). In real life, we are not so lucky, prediction or observation model could become non-linear,

$$\begin{aligned} f x_{k-1} &\rightarrow \mathbf{f}(x_{k-1}) \\ h x_k &\rightarrow \mathbf{h}(x_k) \end{aligned} \quad (6)$$

here I am using $\mathbf{f}/\mathbf{h}$ to indicate they are generic functions so that they are distinguished from constants f/h . It is also possible that the noises are non-Gaussian, or even non-additive. A good example of the later case is stochastic volatility model,

$$x_k = a x_{k-1} + w_{k-1}, \quad w_{k-1} \sim \mathcal{N}(0, 1) \quad (7)$$

$$z_k = b \exp(x_k/2) \cdot v_k, \quad v_k \sim \mathcal{N}(0, 1) \quad (8)$$

where we have pure multiplicative noise for the observation process, and a, b are just some constant. There is really no one-fits-all filter algorithm for the most generic problems. We will start with additive gaussian noise but non-linear observation function, and understand two basic non-linear generalization of Kalman filter, namely Extended Kalman Filter (EKF) and Unscented Kalman Filter (UKF). EKF and UKF will be very important for later applications.

2.2 Algorithm

Recall the most general framework for the filter,

$$\begin{aligned} \textbf{Prediction:} \quad p(x_k \mid z_{1:k-1}) &= \int p(x_k \mid x_{k-1}) p(x_{k-1} \mid z_{1:k-1}) dx_{k-1} \\ \textbf{Update:} \quad p(x_k \mid z_{1:k}) &= \frac{p(z_k \mid x_k) p(x_k \mid z_{1:k-1})}{p(z_k \mid z_{1:k-1})} \end{aligned} \quad (9)$$

Because the process are linear gaussian, the distribution $p(x_k \mid z_{1:k-1})$ and $p(x_k \mid z_{1:k})$ are Gaussian for all time k . To Characterize Gaussian distributions, all we need is the mean and variance. For the prediction and update processes, we keep track of the mean and variances separately.

- m_t^- : predicted mean (prior), P_t^- : predicted variance (prior)
- m_t : updated mean (posterior), P_t : updated variance (posterior)

For any filter, we need an initial condition,

$$p(x_0) = (x_0 \mid m_0, P_0). \quad (10)$$

A process is *stationary* if its statistical properties — mean, variance — do not change over time. For the linear state evolution $x_k = a x_{k-1} + w_{k-1}$, this happens when $|a| < 1$: the state is repeatedly pulled back toward zero, and after many steps the variance settles to a fixed value. We can find this value by asking: if the variance at step k equals the variance at step $k-1$ (call it σ^2), what must σ^2 be?

$$\sigma^2 = a^2 \sigma^2 + Q \quad \Rightarrow \quad \sigma^2 = \frac{Q}{1 - a^2}. \quad (11)$$

This is the variance of the stationary distribution. If the process has been running long before we start observing it — the usual assumption — then at time $k=0$ the state is already drawn from this

equilibrium. The natural initial condition is therefore $m_0 = 0$ and $P_0 = Q/(1 - a^2)$. Starting here avoids a transient period where the filter is far from the true distribution.

When $|a| \geq 1$, there is no equilibrium. The state can drift arbitrarily far from zero and the variance grows without bound over time — the process is *non-stationary*. There is no principled variance to start from, so we express complete uncertainty: $m_0 = 0$ and $P_0 = \kappa$ with κ very large. This is a *diffuse prior* — it says we have no idea where the state is at time zero. As observations arrive, the filter rapidly pulls P_k down toward something sensible, so the exact value of κ does not matter as long as it is large enough to be non-informative.

Now we compute the update rules for the prediction process, given the mean and variance from the previous update step (m_{k-1}, P_{k-1}) , which characterize $p(x_{k-1} | z_{1:k-1})$. We have, for one realization of the prediction

$$x_k^- = f x_{k-1} + \omega_k \quad (12)$$

It is obvious that,

$$\begin{aligned} \mathbb{E}[x_k^- | 1 : z_{k-1}] &= \mathbb{E}[f x_{k-1} + \omega_k | 1 : z_{k-1}] \\ &= f \mathbb{E}[x_{k-1} | 1 : z_{k-1}] \\ &= f m_{k-1} \end{aligned} \quad (13)$$

and for the variation, we just stick to the definition,

$$\begin{aligned} P_k^- &= \text{Var}[x_k | z_{1:k-1}] = \mathbb{E}[x_k^2 | z_{1:k-1}] - (\mathbb{E}[x_k | z_{1:k-1}])^2 \\ &= f^2 \mathbb{E}[x_{k-1}^2 | z_{1:k-1}] - f^2 (\mathbb{E}[x_{k-1} | z_{1:k-1}])^2 + Q \\ &= f^2 P_{k-1} + Q \end{aligned} \quad (14)$$

where the first contribution is just the prediction noise and the second term is the input noise scale by the determinist part of the evolution. After prediction, we need to do correction based on the new observation z_k . By Bayes' rule:

$$p(x_k | z_{1:k}) = \frac{p(z_k | x_k) p(x_k | z_{1:k-1})}{p(z_k | z_{1:k-1})} \propto p(z_k | x_k) p(x_k | z_{1:k-1}). \quad (15)$$

For the mean and variance, we don't have to need the normalization constant. Let's take log on both side.

$$-\frac{(x_k - m_k)^2}{2P_k} = -\frac{(z_k - h x_k)^2}{2R} - \frac{(x_k - m_k^-)^2}{2P_k^-} \quad (16)$$

from this, it is clear to read of the coefficient of x_k^2 to get,

$$\frac{1}{P_k} = \frac{h^2}{R} + \frac{1}{P_k^-} \quad (17)$$

equivalently,

$$P_k = \frac{R P_k^-}{h^2 P_k^- + R} = (1 - k_k h) P_k^- \quad (18)$$

For simplicity, we define the **Kalman gain**:

$$K_k = \frac{P_k^- h}{h^2 P_k^- + R} = P_k^- h / S_k \quad (19)$$

where we also introduce **innovation covariance** $S_k = \frac{1}{h^2 P_k^- + R}$. After some algebra:

$$m_k = m_k^- + K_k v_k \quad (20)$$

where we denote the **innovation** $v_k = z_k - h m_k^-$, which is the difference between observed data and observation of the predicted mean. Let us summarize the algorithm,

Algorithm 1 Kalman Filter (1-d, additive noise)

Require: Model: $x_k = f x_{k-1} + \omega_{k-1}$, $z_k = h x_k + v_k$, $\omega_{k-1} \sim \mathcal{N}(0, Q)$, $v_k \sim \mathcal{N}(0, R)$

Require: Initial conditions: m_0 , P_0

```
1: for  $k = 1, 2, 3, \dots$  do
2:   Prediction:
3:    $m_k^- \leftarrow f m_{k-1}$ 
4:    $P_k^- \leftarrow f^2 P_{k-1} + Q$ 
5:   Update:
6:    $\nu_k \leftarrow z_k - h m_k^-$ 
7:    $S_k \leftarrow h^2 P_k^- + R$ 
8:    $K_k \leftarrow P_k^- h / S_k$ 
9:    $m_k \leftarrow m_k^- + K_k \nu_k$ 
10:   $P_k \leftarrow (1 - K_k h) P_k^-$ 
11: end for
```

Algorithm 2 Kalman Filter (vector, additive noise)

Require: Model: $\mathbf{x}_k = \mathbf{F} \mathbf{x}_{k-1} + \boldsymbol{\omega}_{k-1}$, $\mathbf{z}_k = \mathbf{H} \mathbf{x}_k + \mathbf{v}_k$, $\boldsymbol{\omega}_{k-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$, $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$

Require: Initial conditions: $\mathbf{m}_0$, $\mathbf{P}_0$

```
1: for  $k = 1, 2, 3, \dots$  do
2:   Prediction:
3:    $\mathbf{m}_k^- \leftarrow \mathbf{F} \mathbf{m}_{k-1}$ 
4:    $\mathbf{P}_k^- \leftarrow \mathbf{F} \mathbf{P}_{k-1} \mathbf{F}^\top + \mathbf{Q}$ 
5:   Update:
6:    $\boldsymbol{\nu}_k \leftarrow \mathbf{z}_k - \mathbf{H} \mathbf{m}_k^-$ 
7:    $\mathbf{S}_k \leftarrow \mathbf{H} \mathbf{P}_k^- \mathbf{H}^\top + \mathbf{R}$ 
8:    $\mathbf{K}_k \leftarrow \mathbf{P}_k^- \mathbf{H}^\top \mathbf{S}_k^{-1}$ 
9:    $\mathbf{m}_k \leftarrow \mathbf{m}_k^- + \mathbf{K}_k \boldsymbol{\nu}_k$ 
10:   $\mathbf{P}_k \leftarrow (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_k^-$ 
11: end for
```

2.3 Numerical stability analysis for Kalman filter

If we implement it as it's derived, kalman filter might be numerically unstable. Higher dimension has more failure mode so let us start with 1-d. When the observation noise becomes very small $R \sim 0$, we have $S_k \sim \frac{1}{h^2 P_k^-}$, which leads to,

$$\lim_{R \rightarrow 0} K = \lim_{R \rightarrow 0} \frac{P^- h}{h^2 P^- + R} = \frac{1}{h} \quad (21)$$

With the standard update: $P^+ = (1 - Kh)P^- \approx 0$. But this is very dangerous, because we should always have positive P^+ . With the accumulation of small roundoff errors, $(1 - Kh)$ can become negative. But with Joseph form,

$$P^+ = (I - Kh)P^-(I - Kh) + K^2 R \sim \frac{R}{h^2} \quad (22)$$

Clearly, Joseph form will resolve the issue in 1-d. Joseph form doesn't cure all problems, but it will remove one potential failure in the numerical implementation. We can prove these update formula are mathematically equivalent.

$$\begin{aligned} \mathbf{P}_k &= (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^-(\mathbf{I} - \mathbf{K}\mathbf{H})^\top + \mathbf{K}\mathbf{R}\mathbf{K}^\top \\ &= \mathbf{P}^- - \mathbf{K}\mathbf{H}\mathbf{P}^- - \mathbf{P}^-\mathbf{H}^\top\mathbf{K}^\top + \mathbf{K}\mathbf{H}\mathbf{P}^-\mathbf{H}^\top\mathbf{K}^\top + \mathbf{K}\mathbf{R}\mathbf{K}^\top \\ &= \mathbf{P}^- - \mathbf{K}\mathbf{H}\mathbf{P}^- - \mathbf{P}^-\mathbf{H}^\top\mathbf{K}^\top + \underbrace{\mathbf{K}(\mathbf{H}\mathbf{P}^-\mathbf{H}^\top + \mathbf{R})\mathbf{K}^\top}_{=\mathbf{S}}. \end{aligned}$$

Using $\mathbf{K} = \mathbf{P}^-\mathbf{H}^\top\mathbf{S}^{-1} \implies \mathbf{K}\mathbf{S} = \mathbf{P}^-\mathbf{H}^\top \implies \mathbf{K}\mathbf{S}\mathbf{K}^\top = \mathbf{P}^-\mathbf{H}^\top\mathbf{K}^\top$:

$$\begin{aligned} \mathbf{P}_k &= \mathbf{P}^- - \mathbf{K}\mathbf{H}\mathbf{P}^- - \mathbf{P}^-\mathbf{H}^\top\mathbf{K}^\top + \mathbf{P}^-\mathbf{H}^\top\mathbf{K}^\top \\ &= \mathbf{P}^- - \mathbf{K}\mathbf{H}\mathbf{P}^- = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^-. \end{aligned}$$

Additionally, we will have potential instability when we try to take the inverse of $\mathbf{S}_k^{-1}$. Let's not get into the details of what would cause the failure of $\mathbf{S}_k^{-1}$. But when it fails, it is usually due to that $\mathbf{S}_k$ having large condition number. Let us denote the eigenvalues of $\mathbf{S}_k$ are $\{\lambda_1, \dots, \lambda_d\}$, where d is the dimension of the state space. When $\kappa(\mathbf{S}_k) \gg 1$, small round off error in $\mathbf{S}_k$, which was not exactly and obtained through numerical calculation will leads to huge difference in the inverse, this means that $\mathbf{S}_k^{-1}$ is unreliable.

2.4 Numerical experiments and failure on nonlinear Models

We know that Kalman filter is only exact on linear Gaussian model, but it is still useful to take a look at the result of kalman filter on non-linear model. This is particular useful on higher dimension when we need to inspect the failure mode and invent targeted fixes to improve the filter qualities. Here we run Kalman on range bearing model, and stochastic volatility model.

1. Stochastic volatility model The latent state $x_t \in \mathbb{R}$ represents the log-volatility.

State transition.

$$x_t = \alpha x_{t-1} + \sigma w_t, \quad w_t \sim \mathcal{N}(0, 1), \quad (23)$$

with persistence parameter $\alpha = 0.91$ and volatility-of-volatility $\sigma = 1.0$. The stationary distribution is $x_0 \sim \mathcal{N}(0, \sigma^2/(1 - \alpha^2))$.

Observation model.

$$y_t = \beta \exp\left(\frac{x_t}{2}\right) v_t, \quad v_t \sim \mathcal{N}(0, 1), \quad (24)$$

with scale parameter $\beta = 0.5$. Note that $p(y_t | x_t) = \mathcal{N}(0, \beta^2 e^{x_t})$, so the observation noise is multiplicative and the likelihood is non-Gaussian in x_t . The naive implementation of KF on this model is guarantee to fail, because to force an additive framework on the model, h will be identically zero. The filter just propagates the prior forward with no updates due to $K_k = 0$ for all k .

2. Range bearing model

The state $\mathbf{x}_t = (x_t, y_t)^\top \in \mathbb{R}^2$ represents a 2D position. The state evolves linearly and observations are nonlinear range-bearing measurements from a sensor at position $(x_s, y_s)^\top$.

State transition.

$$\mathbf{x}_t = F \mathbf{x}_{t-1} + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, Q), \quad (25)$$

where $F = I_2$ (constant-position model) and $Q = 0.01 I_2$.

Observation model.

$$\mathbf{z}_t = h(\mathbf{x}_t) + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, R), \quad (26)$$

with the nonlinear measurement function

$$h(\mathbf{x}) = \begin{pmatrix} \sqrt{(x - x_s)^2 + (y - y_s)^2} \\ \text{atan2}(y - y_s, x - x_s) \end{pmatrix}, \quad (27)$$

and diagonal observation noise covariance $R = \text{diag}(\sigma_r^2, \sigma_b^2)$, where $\sigma_r = 0.1$ (range noise std.) and $\sigma_b = 0.1$ rad (bearing noise std.). The sensor is located at the origin $(x_s, y_s) = (0, 0)$.

Initial distribution.

$$\mathbf{x}_0 \sim \mathcal{N}(\boldsymbol{\mu}_0, \Sigma_0), \quad \boldsymbol{\mu}_0 = (1, 1)^\top, \quad \Sigma_0 = I_2. \quad (28)$$

3 EKF and UKF

Previously, we discussed the Kalman filter and saw its failure on non-linear models. Here we introduce generalization of Kalman filter, Extended Kalman Filter (EKF) and Unscented Kalman Filter (UKF) [1], that can handle non-linearity to some extent. They are both designed to incorporate nonlinear corrections in the observation model, namely, $\mathbf{h}(x)$, with additive noises. In the literature, there are improved versions known as augmented EKF/UKF [1] that can handle non-additive noise but these are beyond the scope of the current document. The only non-additive noise model we will run into is the 1-d Stochastic volatility model, and we will use an approximation to turn it into additive noise model and additional improvement on EKF/UKF is overkill for this model.

3.1 EKF

EKF is a nonlinear improvement on KF, in the sense that we still want to somehow preserve most of the KF algorithm. What non-linearity breaks is the linear structure of the model.

$$\begin{aligned}\hat{\mathbf{F}}\mathbf{x}_{k-1} &\rightarrow \mathbf{f}(\mathbf{x}_{k-1}) \\ \hat{\mathbf{H}}\mathbf{x}_k &\rightarrow \mathbf{h}(\mathbf{x}_{k-1})\end{aligned}\tag{29}$$

but what if we expand the prediction and observation model,

$$\begin{aligned}\mathbf{f}(\mathbf{x} - \mathbf{m}_{k-1}) &\approx \mathbf{f}(\mathbf{m}_{k-1}) + \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\mathbf{m}_{k-1}} (\mathbf{x} - \mathbf{m}_{k-1}) + \mathcal{O}((\mathbf{x} - \mathbf{m}_{k-1})^2) \\ \mathbf{h}(\mathbf{x} - \mathbf{m}_k^-) &\approx \mathbf{h}(\mathbf{m}_k^-) + \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\mathbf{m}_k^-} (\mathbf{x} - \mathbf{m}_k^-) + \mathcal{O}((\mathbf{x} - \mathbf{m}_{k-1})^2)\end{aligned}\tag{30}$$

by weak nonlinearity, I meant when the second derivative can be ignored. We would expect such linear approximation to fail when the following two terms are comparable.

$$\left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\mathbf{m}_k^-} (\mathbf{x} - \mathbf{m}_k^-) \sim \left. \frac{\partial^2 \mathbf{h}}{\partial \mathbf{x}^2} \right|_{\mathbf{m}_k^-} (\mathbf{x} - \mathbf{m}_{k-1})^2\tag{31}$$

When linear approximation holds, we can use the Kalman filter algorithm by adding time dependence to the evolution and observation matrices.

$$\begin{aligned}\mathbf{F}_k &\leftarrow \mathbf{f}(\mathbf{m}_{k-1}) \\ \mathbf{H}_k &\leftarrow \mathbf{h}(\mathbf{m}_k^-)\end{aligned}\tag{32}$$

Algorithm 3 Extended Kalman Filter (EKF)

Require: Model: $\mathbf{x}_k = \mathbf{f}(\mathbf{x}_{k-1}) + \boldsymbol{\omega}_{k-1}$, $\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k) + \mathbf{v}_k$, $\boldsymbol{\omega}_{k-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$, $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$

Require: Initial: $\mathbf{m}_0, \mathbf{P}_0$

```

1: for  $k = 1, 2, \dots$  do
2:   Prediction:
3:    $\mathbf{m}_k^- \leftarrow \mathbf{f}(\mathbf{m}_{k-1})$ 
4:    $\mathbf{F} \leftarrow \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\mathbf{m}_{k-1}}$ 
5:    $\mathbf{P}_k^- \leftarrow \mathbf{F} \mathbf{P}_{k-1} \mathbf{F}^\top + \mathbf{Q}$ 
6:   Update :
7:    $\hat{\mathbf{z}} \leftarrow \mathbf{h}(\mathbf{m}_k^-)$ 
8:    $\mathbf{H} \leftarrow \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\mathbf{m}_k^-}$ 
9:    $\boldsymbol{\nu} \leftarrow \mathbf{z}_k - \hat{\mathbf{z}}$  (innovation)
10:   $\mathbf{S} \leftarrow \mathbf{H} \mathbf{P}_k^- \mathbf{H}^\top + \mathbf{R}$ 
11:   $\mathbf{K} \leftarrow \mathbf{P}_k^- \mathbf{H}^\top \mathbf{S}^{-1}$  (Kalman gain via Cholesky solve)
12:   $\mathbf{m}_k \leftarrow \mathbf{m}_k^- + \mathbf{K} \boldsymbol{\nu}$ 
13:   $\mathbf{P}_k \leftarrow (\mathbf{I} - \mathbf{K} \mathbf{H}) \mathbf{P}_k^- (\mathbf{I} - \mathbf{K} \mathbf{H})^\top + \mathbf{K} \mathbf{R} \mathbf{K}^\top$  (Joseph form)
14: end for
```

3.2 UKF

The EKF approximates the nonlinear model by linearization, which requires computing the Jacobians $\mathbf{F}$ and $\mathbf{H}$. The UKF takes a different route: instead of approximating the *function*, it approximates the *distribution* that results from pushing a Gaussian through the function. The key observation is that the Kalman gain can be written without any reference to $\mathbf{H}$. In the linear case, $\mathbf{K} = \mathbf{P}\mathbf{H}^\top\mathbf{S}^{-1}$ and $\mathbf{z}_k^- = \mathbf{H}\mathbf{x}_k^- + \mathbf{v}_k$ with $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$ independent of $\mathbf{x}_k^-$. The cross-covariance between the predicted state and predicted observation is

$$\begin{aligned}\text{Cov}(\mathbf{x}_k^-, \mathbf{z}_k^-) &= \mathbb{E}[(\mathbf{x}_k^- - \mathbf{m}_k^-)(\mathbf{z}_k^- - \mathbf{H}\mathbf{m}_k^-)^\top] \\ &= \mathbb{E}[(\mathbf{x}_k^- - \mathbf{m}_k^-)(\mathbf{H}(\mathbf{x}_k^- - \mathbf{m}_k^-) + \mathbf{v}_k)^\top] \\ &= \mathbb{E}[(\mathbf{x}_k^- - \mathbf{m}_k^-)(\mathbf{x}_k^- - \mathbf{m}_k^-)^\top] \mathbf{H}^\top \\ &= \mathbf{P}_k^- \mathbf{H}^\top,\end{aligned}\tag{33}$$

where the cross term $\mathbb{E}[(\mathbf{x}_k^- - \mathbf{m}_k^-)\mathbf{v}_k^\top] = 0$ by independence. So $\mathbf{P}_k^- \mathbf{H}^\top = \text{Cov}(\mathbf{x}_k^-, \mathbf{z}_k^-) \equiv \mathbf{P}_{xz}$. For the innovation covariance,

$$\begin{aligned}\text{Cov}(\mathbf{z}_k^-, \mathbf{z}_k^-) &= \mathbb{E}[(\mathbf{H}(\mathbf{x}_k^- - \mathbf{m}_k^-) + \mathbf{v}_k)(\mathbf{H}(\mathbf{x}_k^- - \mathbf{m}_k^-) + \mathbf{v}_k)^\top] \\ &= \mathbf{H} \mathbb{E}[(\mathbf{x}_k^- - \mathbf{m}_k^-)(\mathbf{x}_k^- - \mathbf{m}_k^-)^\top] \mathbf{H}^\top + \mathbb{E}[\mathbf{v}_k \mathbf{v}_k^\top] \\ &= \mathbf{H} \mathbf{P}_k^- \mathbf{H}^\top + \mathbf{R} = \mathbf{S},\end{aligned}\tag{34}$$

where the cross terms again vanish by independence. So $\text{Cov}(\mathbf{z}_k^-, \mathbf{z}_k^-) \equiv \mathbf{P}_{zz} = \mathbf{S}$. The Kalman gain therefore takes the equivalent form

$$\mathbf{K} = \mathbf{P}_{xz} \mathbf{S}^{-1},\tag{35}$$

and this expression makes no assumption about linearity and is "blind" to the functional form of the models. If we can estimate $\mathbf{P}_{xz}$ and $\mathbf{S}$ for a nonlinear $\mathbf{h}$, we can run the Kalman update as-is.

The question is how to compute these covariances when $\mathbf{h}$ is nonlinear. EKF evaluates them analytically via the Jacobian and discards higher-order terms. UKF instead draws $2d + 1$ *sigma points* deterministically from the predicted Gaussian $\mathcal{N}(\mathbf{m}_k^-, \mathbf{P}_k^-)$, propagates each through $\mathbf{h}$, and computes the covariances as weighted sums:

$$\hat{\mathbf{z}} = \sum_j W_m^{(j)} \mathbf{h}(\boldsymbol{\chi}_j),\tag{36}$$

$$\mathbf{S} = \sum_j W_c^{(j)} (\mathbf{h}(\boldsymbol{\chi}_j) - \hat{\mathbf{z}})(\mathbf{h}(\boldsymbol{\chi}_j) - \hat{\mathbf{z}})^\top + \mathbf{R},\tag{37}$$

$$\mathbf{P}_{xz} = \sum_j W_c^{(j)} (\boldsymbol{\chi}_j - \mathbf{m}_k^-)(\mathbf{h}(\boldsymbol{\chi}_j) - \hat{\mathbf{z}})^\top.\tag{38}$$

The sigma points are chosen so that their weighted mean and covariance exactly match the Gaussian $\mathcal{N}(\mathbf{m}_k^-, \mathbf{P}_k^-)$, which guarantees the approximation is at least second-order accurate (capturing the curvature of $\mathbf{h}$ that EKF misses). No Jacobian is required. With $\mathbf{P}_{xz}$ and $\mathbf{S}$ in hand, the rest of the update is identical to the Kalman filter.

Algorithm 4 Unscented Kalman Filter (UKF)

Require: Same model as EKF. UKF parameters: α, β, κ ; $\lambda = \alpha^2(d + \kappa) - d$

Require: UKF weights: $W_m^{(0)} = \frac{\lambda}{d + \lambda}$, $W_c^{(0)} = W_m^{(0)} + 1 - \alpha^2 + \beta$, $W_m^{(j)} = W_c^{(j)} = \frac{1}{2(d + \lambda)}$ for $j = 1, \dots, 2d$

```
1: for  $k = 1, 2, \dots$  do
2:   Prediction:
3:   Generate  $2d + 1$  sigma points:  $\chi_0 = \mathbf{m}_{k-1}$ 
4:    $\chi_j = \mathbf{m}_{k-1} + \left[ \sqrt{(d + \lambda) \mathbf{P}_{k-1}} \right]_j$ ,  $j = 1, \dots, d$ 
5:    $\chi_{d+j} = \mathbf{m}_{k-1} - \left[ \sqrt{(d + \lambda) \mathbf{P}_{k-1}} \right]_j$ ,  $j = 1, \dots, d$ 
6:   Propagate:  $\chi_j^* \leftarrow \mathbf{f}(\chi_j)$  for all  $j$ 
7:    $\mathbf{m}_k^- \leftarrow \sum_{j=0}^{2d} W_m^{(j)} \chi_j^*$ 
8:    $\mathbf{P}_k^- \leftarrow \sum_{j=0}^{2d} W_c^{(j)} (\chi_j^* - \mathbf{m}_k^-)(\chi_j^* - \mathbf{m}_k^-)^\top + \mathbf{Q}$ 
9:   Update (for each particle  $i$ ):
10:  Regenerate sigma points from  $(\mathbf{m}_k^-, \mathbf{P}_k^-)$ 
11:  Propagate through observation:  $\gamma_j \leftarrow \mathbf{h}(\chi_j)$ 
12:   $\hat{\mathbf{z}} \leftarrow \sum_{j=0}^{2d} W_m^{(j)} \gamma_j$ 
13:   $\mathbf{S} \leftarrow \sum_{j=0}^{2d} W_c^{(j)} (\gamma_j - \hat{\mathbf{z}})(\gamma_j - \hat{\mathbf{z}})^\top + \mathbf{R}$ 
14:   $\mathbf{P}_{xz} \leftarrow \sum_{j=0}^{2d} W_c^{(j)} (\chi_j - \mathbf{m}_k^-)(\gamma_j - \hat{\mathbf{z}})^\top$ 
15:   $\mathbf{K} \leftarrow \mathbf{P}_{xz} \mathbf{S}^{-1}$  (Kalman gain via Cholesky solve)
16:   $\mathbf{m}_k \leftarrow \mathbf{m}_k^- + \mathbf{K}(\mathbf{z}_k - \hat{\mathbf{z}})$ 
17:   $\mathbf{P}_k \leftarrow \mathbf{P}_k^- - \mathbf{K} \mathbf{P}_{xz}^\top - \mathbf{P}_{xz} \mathbf{K}^\top + \mathbf{K} \mathbf{S} \mathbf{K}^\top$ 
18: end for
```

Here d is the *state dimension*. The number of sigma points $2d + 1$ is fixed by the method — one at the mean, d pairs placed symmetrically along each column of $\sqrt{(d + \lambda) \mathbf{P}}$.

The three tuning parameters α, β, κ all feed into the composite scaling parameter $\lambda = \alpha^2(d + \kappa) - d$. The sigma point displacement is $h = \sqrt{(d + \lambda) \mathbf{P}}$, so λ directly sets the spread. $\alpha \in (0, 1]$ is the primary knob. The typical default $\alpha = 10^{-3}$ with $\kappa = 0$ gives $\lambda \approx -d$, so $(d + \lambda) \approx \alpha^2 d \approx 0$ — the sigma points collapse nearly to the mean and the filter degenerates toward a linearization. κ is minor and usually left at zero.

β enters only in $W_c^{(0)} = W_m^{(0)} + (1 - \alpha^2 + \beta)$, adjusting how much the central sigma point contributes to the covariance. To see why $\beta = 2$ is optimal for Gaussians, consider the simplest case: $d = 1$, $f(x) = x^2$, $x \sim \mathcal{N}(0, P)$. The true variance is $\text{Var}[x^2] = 3P^2 - P^2 = 2P^2$ (Gaussian fourth moment $\mathbb{E}[x^4] = 3P^2$). With $\kappa = 0$, let $h^2 = (1 + \lambda)P$; the three sigma points give $f(\chi_j) \in \{0, h^2, h^2\}$. The UKF mean $\hat{z} = 2W_m^{(1)}h^2 = P$ is already exact. The covariance estimate is

$$\widehat{\text{Var}} = W_c^{(0)}(0 - P)^2 + 2W_c^{(1)}(h^2 - P)^2 = \left(W_c^{(0)} + \frac{\lambda^2}{1 + \lambda} \right) P^2. \quad (39)$$

Setting this equal to $2P^2$ requires $W_c^{(0)} + \lambda^2/(1 + \lambda) = 2$. Substituting $W_c^{(0)} = \lambda/(1 + \lambda) + (1 - \alpha^2 + \beta)$ and $\lambda = \alpha^2 - 1$ (for $d = 1$, $\kappa = 0$):

$$(\alpha^2 - 1) + (1 - \alpha^2 + \beta) = 2 \implies \beta = 2. \quad (40)$$

The α^2 terms cancel exactly, so $\beta = 2$ holds for any α when $d = 1$, $\kappa = 0$. For $d > 1$ the cancellation is approximate, but $\beta = 2$ remains a good default.

Parameter choice for the SV model. The 1D stochastic volatility model has $d = 1$, so there are always 3 sigma points. With the default $\alpha = 10^{-3}$, $\kappa = 0$: $\lambda \approx -1$, giving $(d + \lambda) \approx 10^{-6}$ — sigma points essentially at the mean, which is too tight for $h(x) = e^{x/2}$. A better choice is $\alpha = 1$, which gives $\lambda = 0$ and places sigma points at $m_k^- \pm \sqrt{P_k^-}$, covering one standard deviation of the predicted distribution and capturing the relevant curvature of $e^{x/2}$. $\beta = 2$ and $\kappa = 0$ are kept as defaults. [add experiments for this](#)

3.3 EKF/UKF on non-linear models

It is straightforward to run EKF and UKF on range bearing. But to make them work for Stochastic Volatility, we do the following trick on the model itself. Recall the model definition,

State transition.

$$x_t = \alpha x_{t-1} + \sigma w_t, \quad w_t \sim \mathcal{N}(0, 1), \quad (41)$$

with persistence parameter $\alpha = 0.91$ and volatility-of-volatility $\sigma = 1.0$. The stationary distribution is $x_0 \sim \mathcal{N}(0, \sigma^2/(1 - \alpha^2))$.

Observation model.

$$y_t = \beta \exp\left(\frac{x_t}{2}\right) v_t, \quad v_t \sim \mathcal{N}(0, 1), \quad (42)$$

with scale parameter $\beta = 0.5$. Note that $p(y_t | x_t) = \mathcal{N}(0, \beta^2 e^{x_t})$. We introduce a log-squared transform $z_t = \log y_t^2$ on the observation model which yields,

$$z_t = \log \beta^2 + x_t + \varepsilon_t, \quad (43)$$

where $\varepsilon_t = \log v_t^2 \sim \log \chi_1^2$ is approximated as $\mathcal{N}(\mu_\varepsilon, \sigma_\varepsilon^2)$ with $\mu_\varepsilon = \psi(1/2) + \log 2 \approx -1.2704$ and $\sigma_\varepsilon^2 = \pi^2/2 \approx 4.935$, where ψ is the digamma function. This linearises the observation in x_t .

4 Particle-Based Methods

4.1 Importance Sampling and Sequential Monte Carlo

The prediction and update integrals from Section 1,

$$\begin{aligned} \textbf{Prediction:} \quad p(x_k \mid z_{1:k-1}) &= \int p(x_k \mid x_{k-1}) p(x_{k-1} \mid z_{1:k-1}) dx_{k-1} \\ \textbf{Update:} \quad p(x_k \mid z_{1:k}) &= \frac{p(z_k \mid x_k) p(x_k \mid z_{1:k-1})}{p(z_k \mid z_{1:k-1})}, \end{aligned} \quad (44)$$

are intractable in general because the prediction integral has no closed form when f or h are nonlinear or the noise is non-Gaussian, and the normalization constant $p(z_k \mid z_{1:k-1})$ in the update is itself an integral over all of state space. The key idea of particle-based methods is to *represent the filtering distribution by a set of weighted samples* (particles). Given N particles $\{x_k^{(i)}\}_{i=1}^N$ with associated weights $\{w_k^{(i)}\}_{i=1}^N$ satisfying $\sum_i w_k^{(i)} = 1$, we approximate

$$p(x_k \mid z_{1:k}) \approx \sum_{i=1}^N w_k^{(i)} \delta(x_k - x_k^{(i)}). \quad (45)$$

With this empirical representation, any posterior expectation reduces to a weighted sum,

$$\mathbb{E}[g(x_k) \mid z_{1:k}] = \int g(x_k) p(x_k \mid z_{1:k}) dx_k \approx \sum_{i=1}^N w_k^{(i)} g(x_k^{(i)}), \quad (46)$$

so recovering the posterior mean and covariance requires no further integration.

Importance Sampling

The challenge is that we generally cannot sample directly from the target $p(x_k \mid z_{1:k})$. *Importance sampling* (IS) resolves this by drawing from a tractable *proposal distribution* $q(x_k)$ and correcting for the mismatch with weights. For any function g ,

$$\mathbb{E}_p[g(x_k)] = \int g(x_k) \frac{p(x_k \mid z_{1:k})}{q(x_k)} q(x_k) dx_k \approx \sum_{i=1}^N w^{(i)} g(x_k^{(i)}), \quad x_k^{(i)} \sim q(x_k), \quad (47)$$

where the *importance weight* is proportional to the ratio $w^{(i)} \propto p(x_k^{(i)} \mid z_{1:k})/q(x_k^{(i)})$. Since $p(x_k \mid z_{1:k})$ is known only up to a normalizing constant, we use the *self-normalized* estimator with unnormalized weights $\tilde{w}^{(i)} \propto p(x_k^{(i)} \mid z_{1:k})/q(x_k^{(i)})$,

$$w_k^{(i)} = \frac{\tilde{w}_k^{(i)}}{\sum_{j=1}^N \tilde{w}_k^{(j)}}. \quad (48)$$

This is consistent as $N \rightarrow \infty$ for any proposal q with support containing that of p .

Sequential Monte Carlo

The real power of IS emerges when it is applied *sequentially*. Suppose at time $k-1$ we hold a particle approximation $\{x_{k-1}^{(i)}, w_{k-1}^{(i)}\}$ of $p(x_{k-1} \mid z_{1:k-1})$. Our *only* goal at time k is the posterior $p(x_k \mid z_{1:k})$, whose mean m_k and covariance P_k are the quantities of interest. The intermediate predicted distribution $p(x_k \mid z_{1:k-1})$ and its moments m_k^-, P_k^- are not ends in themselves—they are merely one route to the posterior. This gives us great freedom: we may draw new particles from *any* proposal

$$x_k^{(i)} \sim q(x_k \mid x_{k-1}^{(i)}, z_k), \quad i = 1, \dots, N, \quad (49)$$

provided it has sufficient support, and correct for the mismatch via IS weights. Writing the target as $p(x_k \mid z_{1:k}) \propto p(z_k \mid x_k) p(x_k \mid x_{k-1}^{(i)})$, the unnormalized weight for each particle is

$$\tilde{w}_k^{(i)} = w_{k-1}^{(i)} \cdot \frac{p(z_k \mid x_k^{(i)}) p(x_k^{(i)} \mid x_{k-1}^{(i)})}{q(x_k^{(i)} \mid x_{k-1}^{(i)}, z_k)}, \quad (50)$$

which after normalization gives $p(x_k | z_{1:k}) \approx \sum_i w_k^{(i)} \delta(x_k - x_k^{(i)})$. This is the *Sequential Monte Carlo* (SMC) framework: the prediction and update integrals are handled together in a single IS step, and the two-step predict-then-update decomposition of the Kalman family is not fundamental here—it is just one way to factor the joint density.

The proposal q is a free design choice, and this freedom is the key to all the methods we study. The simplest option $q(x_k | x_{k-1}^{(i)}, z_k) = p(x_k | x_{k-1}^{(i)})$ ignores the current observation entirely, simplifying (50) to $\tilde{w}_k^{(i)} \propto w_{k-1}^{(i)} \cdot p(z_k | x_k^{(i)})$ —this is the Bootstrap Particle Filter. More sophisticated methods either use a smarter proposal that already incorporates z_k (while still relying on weights to correct for the approximation), or abandon importance weights altogether and instead move particles deterministically toward the posterior using a flow constructed from a Gaussian approximation to $p(x_k | z_{1:k})$ —as in the EDH and LEDH flow filters described later.

A well-known practical issue with weighted SMC is *weight degeneracy*: after many steps, the multiplicative weight update concentrates nearly all weight on very few particles. The standard remedy is *resampling*, and we will study its effect by applying Bootstrap Particle Filter on non-linear models.

4.2 Bootstrap Particle Filter

4.2.1 Algorithm

The BPF is the simplest instantiation of SMC: the proposal is the transition prior $q = p(x_k | x_{k-1}^{(i)})$, so each particle is propagated by forward-simulating the dynamics and scored against the new observation. From (50) the weight reduces to $\tilde{w}_k^{(i)} \propto w_{k-1}^{(i)} \cdot p(z_k | x_k^{(i)})$.

Algorithm 5 Bootstrap Particle Filter (BPF)

Require: particles $\{x_{k-1}^{(i)}, w_{k-1}^{(i)}\}_{i=1}^N$, observation z_k

1: **for** $i = 1$ **to** N **do**

2: **Predict:** sample $x_k^{(i)} \sim p(x_k | x_{k-1}^{(i)})$

3: **Weight:** $\tilde{w}_k^{(i)} \leftarrow w_{k-1}^{(i)} \cdot p(z_k | x_k^{(i)})$

4: **end for**

5: **Normalize:** $w_k^{(i)} \leftarrow \tilde{w}_k^{(i)} / \sum_{j=1}^N \tilde{w}_k^{(j)}$ for all i

6: **Estimate:** $m_k = \sum_i w_k^{(i)} x_k^{(i)}$, $P_k = \sum_i w_k^{(i)} (x_k^{(i)} - m_k)(x_k^{(i)} - m_k)^\top$

7: **if** $N_{\text{eff}} = (\sum_i (w_k^{(i)})^2)^{-1} < N/2$ **then**

8: **Resample** (Algorithm 6); set $w_k^{(i)} \leftarrow 1/N$ for all i

9: **end if**

Ensure: $\{x_k^{(i)}, w_k^{(i)}\}_{i=1}^N \approx p(x_k | z_{1:k})$, moments m_k, P_k

4.2.2 Particle Degeneracy and Resampling

Because the weight update multiplies by a likelihood $p(z_k | x_k^{(i)}) \leq 1$ at every step, weights collapse geometrically: after T steps nearly all probability mass concentrates on one or two particles. The *effective sample size*

$$N_{\text{eff}} = \frac{1}{\sum_{i=1}^N (w_k^{(i)})^2} \quad (51)$$

quantifies this: it equals N when all weights are uniform and drops toward 1 at full collapse. We trigger resampling when $N_{\text{eff}} < N/2$.

Multinomial resampling. The conceptually simplest fix draws N independent samples from the categorical distribution defined by the current weights,

$$x_k^{(i)} \sim \text{Categorical}(\{w_k^{(j)}\}_{j=1}^N), \quad i = 1, \dots, N, \quad (52)$$

then resets all weights to $1/N$. High-weight particles are duplicated; low-weight ones are discarded. While unbiased, this introduces additional Monte Carlo variance: because each draw is independent, particles with moderate weight may be selected zero times by chance. The cost is $O(N \log N)$ via the inverse-CDF method.

Systematic resampling (the actual implementation). We use systematic resampling, which addresses both drawbacks. A single draw $u \sim \mathcal{U}(0, 1/N)$ places N evenly-spaced points across $[0, 1]$:

$$u^{(i)} = u + \frac{i-1}{N}, \quad i = 1, \dots, N. \quad (53)$$

Particle j is selected for slot i if $u^{(i)}$ falls in $(\sum_{l < j} w_k^{(l)}, \sum_{l \leq j} w_k^{(l)}]$. Because the grid is deterministically spread over $[0, 1]$, every particle with weight $w^{(j)} > 1/N$ is guaranteed to appear at least $\lfloor Nw^{(j)} \rfloor$ times, and the copy count deviates from the ideal $Nw^{(j)}$ by at most 1. This eliminates the random clustering of multinomial resampling, minimizes the variance of the resampling step, and reduces the cost to $O(N)$ via a single linear pass through the cumulative weight sum.

Algorithm 6 Systematic Resampling

Require: weights $\{w^{(i)}\}_{i=1}^N$, positions $\{x^{(i)}\}_{i=1}^N$
1: Draw $u \sim \mathcal{U}(0, 1/N)$; set $u^{(i)} = u + (i-1)/N$ for $i = 1, \dots, N$
2: Compute cumulative sums $C_j = \sum_{l=1}^j w^{(l)}$
3: **for** $i = 1$ **to** N **do**
4: Find j_i such that $C_{j_i-1} < u^{(i)} \leq C_{j_i}$; set $\bar{x}^{(i)} \leftarrow x^{(j_i)}$
5: **end for**
Ensure: $\{\bar{x}^{(i)}, 1/N\}_{i=1}^N$

After resampling the particle set has uniform weights and represents the same distribution as before, but with diversity restored. The cost is *sample impoverishment*: multiplicity alone now encodes the distribution, with no weight diversity. This motivates the flow-based methods described next, which avoid resampling altogether by moving particles directly toward the posterior.

4.3 EDH/LEDH Flow Filter

[2, 3, 4]

4.3.1 Use Fokker-Planck to reach the target distribution

Recall the update step,

$$p(x_k \mid z_{1:k}) \propto \underbrace{p(x_k \mid z_{1:k-1})}_{g(x)} \cdot \underbrace{p(z_k \mid x_k)}_{h(x)}. \quad (54)$$

The Bootstrap Particle Filter handles this by reweighting: sample from $g(x)$, then multiply each particle's weight by $h(x^{(i)})$. That multiplication is precisely what causes weight degeneracy. Flow filters [2, 3] take a different route: instead of adjusting weights, we move the particles. If we can find a deterministic map that transports samples from $g(x)$ into samples from $g(x)h(x)$, we have solved the update step exactly—with no weights and no degeneracy.

To do this continuously, introduce a pseudo-time $\lambda \in [0, 1]$ and the *log-homotopy*:

$$\log p(x, \lambda) = \log g(x) + \lambda \log h(x). \quad (55)$$

At $\lambda = 0$, $p(x, 0) = g(x)$ is the predicted distribution; at $\lambda = 1$, $p(x, 1) \propto g(x)h(x)$ is the posterior. We want a flow $f(x, \lambda)$ such that particles obeying

$$\frac{d\mathbf{x}}{d\lambda} = f(\mathbf{x}, \lambda) \quad (56)$$

follow this homotopy path from prior to posterior.

A deterministic flow induces a density evolution governed by the zero-diffusion *Fokker-Planck equation* (also called the continuity equation for probability conservation),

$$\frac{\partial p}{\partial \lambda} = -\nabla \cdot (pf). \quad (57)$$

The homotopy requires $\frac{\partial \log p}{\partial \lambda} = \log h(x)$, which together with the Fokker-Planck equation (dividing by p and using $\nabla \cdot (pf) = p\nabla \cdot f + (\nabla p) \cdot f$) yields the fundamental constraint:

$$\boxed{\log h(x) = -\nabla \cdot f - (\nabla \log p) \cdot f} \quad (58)$$

This is a PDE in the unknown vector field f , and it has infinitely many solutions. Any flow satisfying this equation will correctly transport particles from $g(x)$ to $g(x)h(x)$. The choice of f is a design decision, completely analogous to the choice of proposal in weighted SMC. We just need to pick one that is tractable to compute.

4.3.2 Determine the evolution kernel

The simplest structured choice is an *affine flow*:

$$f(x, \lambda) = A(\lambda)x + b(\lambda), \quad (59)$$

where $A(\lambda)$ and $b(\lambda)$ are a matrix and vector to be determined. A stretches and rotates particles while b translates them. Crucially, they are not free parameters — they must be chosen so that the flow satisfies the Fokker-Planck constraint. For an affine flow $\nabla \cdot f = \text{Tr}(A)$, and the constraint becomes algebraic once we know $\nabla \log p(x, \lambda)$.

This is where we need an assumption. To evaluate $\nabla \log p(x, \lambda)$ in the constraint, we need to know what $p(x, \lambda)$ looks like for intermediate λ . Different assumptions lead to different flows — different schedules for $A(\lambda)$ and $b(\lambda)$ — but they all share the same endpoints: $\lambda = 0$ is the prior $g(x)$ and $\lambda = 1$ is the posterior $g(x)h(x)$, fixed by the homotopy construction. The assumption only determines the path taken between those two endpoints; it does not change the destination.

We assume $p(x, \lambda)$ is Gaussian:

$$p(x, \lambda) \sim \mathcal{N}(\bar{\eta}_\lambda, P_\lambda), \quad (60)$$

with precision and mean evolving as

$$P_\lambda^{-1} = P^{-1} + \lambda H^T R^{-1} H, \quad (61)$$

$$\bar{\eta}_\lambda = P_\lambda [P^{-1} \bar{\eta}_0 + \lambda H^T R^{-1} z]. \quad (62)$$

This is the only role of the Gaussian assumption: it gives a tractable expression for $\nabla \log p(x, \lambda) = -P_\lambda^{-1}(x - \bar{\eta}_\lambda)$, which we substitute into the constraint to solve for A and b . A different assumption about the intermediate distribution would yield a different flow schedule, but the particles would still travel between the same prior and posterior.

With the Gaussian form, $\nabla \log p(x, \lambda) = -P_\lambda^{-1}(x - \bar{\eta}_\lambda)$. Differentiating the precision with respect to λ and using the matrix identity $dP_\lambda/d\lambda = -P_\lambda(dP_\lambda^{-1}/d\lambda)P_\lambda$ gives

$$\frac{dP_\lambda}{d\lambda} = -P_\lambda H^T R^{-1} H P_\lambda. \quad (63)$$

Matching this with the covariance evolution under the affine flow, $AP_\lambda + P_\lambda A^T = dP_\lambda/d\lambda$, and solving the resulting Riccati equation yields:

$$A(\lambda) = -\frac{1}{2}P_\lambda H^T R^{-1} H = -\frac{1}{2}P H^T (\lambda H P H^T + R)^{-1} H \quad (64)$$

For $b(\lambda)$, requiring that the mean evolves correctly under the flow gives:

$$b(\lambda) = (I + 2\lambda A) [(I + \lambda A) P H^T R^{-1} z + A \bar{\eta}_0] \quad (65)$$

These two equations define the EDH flow completely. The log-homotopy definition forces λ to run from 0 to 1, so the particle integration over λ is a finite integral solved numerically with small steps ε_j .

4.3.3 EDH vs. LEDH: incremental improvements in linearization quality.

All variants use exactly the same framework: the same log-homotopy, the same affine ansatz, the same Gaussian assumption for $\nabla \log p$. They differ only in *where and when* the Jacobian $H = \partial h / \partial x$ is evaluated. There are three levels of refinement:

- **EDH (fixed global linearization):** H is computed once at the global predicted mean $\bar{\eta}_0$ at the start of the flow ($\lambda = 0$) and held fixed for the entire λ integration. All particles share the same $A(\lambda)$ and $b(\lambda)$ at every step. This is the most efficient variant but the least accurate: as particles move during the flow, the mean $\bar{\eta}(\lambda)$ drifts away from $\bar{\eta}_0$, yet the linearization never corrects for this.

- **EDH (intermediate re-linearization):** At each λ step, H is re-evaluated at the current global mean $\bar{\eta}^{(\lambda)}$, and the nonlinear correction $e = h(\bar{\eta}^{(\lambda)}) - H\bar{\eta}^{(\lambda)}$ is computed. The drift vector b is updated to use $(z - e)$ instead of z , capturing the residual between the true observation function and its local linearization. The wind field is still uniform across particles — all particles share the same A and b at each step — but both are recomputed at each step to track where the cloud has moved. This is strictly richer than the fixed variant: not only does H improve, but b also corrects for the nonlinearity of h at the current operating point.
- **LEDH (local, per-particle linearization) [4]:** $H^{(i)} = \partial h / \partial x|_{x^{(i)}(\lambda)}$ is computed at each particle's own current position at each λ step, giving every particle its own $A^{(i)}(\lambda)$ and $b^{(i)}(\lambda)$. The wind field is now particle-specific: each leaf is blown by a wind computed from local conditions. The linearization error is controlled by the distance from $x^{(i)}$ to the linearization point—which is now zero. This is the most accurate variant, at the cost of N Jacobian evaluations per λ step.

Each step in this hierarchy adds one more correction to where the linearization is taken—across particles, across pseudo-time, or both. None of them is a different algorithm. They are the same affine flow with progressively better approximations to $\nabla \log p$ plugged in.

4.3.4 Algorithm

All three variants share the same structural role for the global EKF/UKF: it runs in parallel with the particle cloud *solely* to supply the covariance $\mathbf{P}$ and the linearization point needed to compute the flow kernel $A(\lambda), b(\lambda)$. It is never used to produce filter estimates. The reported output—the posterior mean m_k and covariance P_k —is always taken from the empirical particle ensemble:

$$m_k = \frac{1}{N} \sum_i \boldsymbol{\eta}^{(i)}, \quad P_k = \frac{1}{N} \sum_i (\boldsymbol{\eta}^{(i)} - m_k)(\boldsymbol{\eta}^{(i)} - m_k)^\top. \quad (66)$$

To keep the global filter from drifting away from the particle cloud, a *feedback step* resets the EKF/UKF mean to the current empirical mean at the start of each time step before prediction. The EKF/UKF covariance $\mathbf{P}$ is then used as the kernel covariance, but the covariance estimate reported to the user is the empirical one above.

Algorithm 7 EDH Flow Filter (Fixed Global Linearization)

Require: Particles $\{\boldsymbol{\eta}^{(i)}\}_{i=1}^N$, global EKF/UKF state $(\bar{\boldsymbol{\eta}}, \mathbf{P})$

Require: Lambda schedule $\{\varepsilon_j\}_{j=1}^{n_\lambda}$

- 1: **for** $k = 1, 2, \dots, T$ **do**
 - 2: **Feedback:** $\bar{\boldsymbol{\eta}} \leftarrow \frac{1}{N} \sum_i \boldsymbol{\eta}^{(i)}$ (reset global mean to empirical mean)
 - 3: **Predict (EKF/UKF):** $(\bar{\boldsymbol{\eta}}_0, \mathbf{P}) \leftarrow \text{EKF/UKF predict}(\bar{\boldsymbol{\eta}}, \mathbf{P})$
 - 4: **Predict (particles):** $\boldsymbol{\eta}^{(i)} \sim p(\mathbf{x}_k | \boldsymbol{\eta}^{(i)})$ for $i = 1, \dots, N$
 - 5: $\mathbf{H} \leftarrow \frac{\partial h}{\partial \mathbf{x}}|_{\bar{\boldsymbol{\eta}}_0}$ (**fixed:** linearize once at $\bar{\boldsymbol{\eta}}_0$, held for all λ steps)
 - 6: $\lambda \leftarrow 0$
 - 7: **for** $j = 1, 2, \dots, n_\lambda$ **do**
 - 8: $\lambda \leftarrow \lambda + \varepsilon_j$
 - 9: $\mathbf{A} \leftarrow -\frac{1}{2} \mathbf{P} \mathbf{H}^\top (\lambda \mathbf{H} \mathbf{P} \mathbf{H}^\top + \mathbf{R})^{-1} \mathbf{H}$
 - 10: $\mathbf{b} \leftarrow (\mathbf{I} + 2\lambda \mathbf{A})[(\mathbf{I} + \lambda \mathbf{A}) \mathbf{P} \mathbf{H}^\top \mathbf{R}^{-1} \mathbf{z}_k + \mathbf{A} \bar{\boldsymbol{\eta}}_0]$
 - 11: $\boldsymbol{\eta}^{(i)} \leftarrow \boldsymbol{\eta}^{(i)} + \varepsilon_j (\mathbf{A} \boldsymbol{\eta}^{(i)} + \mathbf{b})$ for all i
 - 12: **end for**
 - 13: **Update (EKF/UKF):** $(\bar{\boldsymbol{\eta}}, \mathbf{P}) \leftarrow \text{EKF/UKF update}(\bar{\boldsymbol{\eta}}_0, \mathbf{P}, \mathbf{z}_k)$ (kernel only; not reported)
 - 14: **Output:** $m_k = \frac{1}{N} \sum_i \boldsymbol{\eta}^{(i)}, \quad P_k = \frac{1}{N} \sum_i (\boldsymbol{\eta}^{(i)} - m_k)(\boldsymbol{\eta}^{(i)} - m_k)^\top$
 - 15: **end for**
-

Algorithm 8 EDH Flow Filter (Intermediate Re-linearization)

Require: Same setup as Algorithm 7

```

1: for  $k = 1, 2, \dots, T$  do
2:   Feedback, Predict (EKF/UKF), Predict (particles): same as Algorithm 7
3:    $\bar{\boldsymbol{\eta}}^{(\lambda)} \leftarrow \bar{\boldsymbol{\eta}}_0$  (track current global mean through flow)
4:    $\lambda \leftarrow 0$ 
5:   for  $j = 1, 2, \dots, n_\lambda$  do
6:      $\lambda \leftarrow \lambda + \varepsilon_j$ 
7:      $\mathbf{H} \leftarrow \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \Big|_{\bar{\boldsymbol{\eta}}^{(\lambda)}}$  (re-linearize at current global mean  $\bar{\boldsymbol{\eta}}^{(\lambda)}$ )
8:      $\mathbf{e} \leftarrow \mathbf{h}(\bar{\boldsymbol{\eta}}^{(\lambda)}) - \mathbf{H}\bar{\boldsymbol{\eta}}^{(\lambda)}$  (nonlinear correction at current mean)
9:      $\mathbf{A} \leftarrow -\frac{1}{2}\mathbf{P}\mathbf{H}^\top(\lambda\mathbf{H}\mathbf{P}\mathbf{H}^\top + \mathbf{R})^{-1}\mathbf{H}$ 
10:     $\mathbf{b} \leftarrow (\mathbf{I} + 2\lambda\mathbf{A})[(\mathbf{I} + \lambda\mathbf{A})\mathbf{P}\mathbf{H}^\top\mathbf{R}^{-1}(\mathbf{z}_k - \mathbf{e}) + \mathbf{A}\bar{\boldsymbol{\eta}}_0]$ 
11:     $\bar{\boldsymbol{\eta}}^{(\lambda)} \leftarrow \bar{\boldsymbol{\eta}}^{(\lambda)} + \varepsilon_j(\mathbf{A}\bar{\boldsymbol{\eta}}^{(\lambda)} + \mathbf{b})$  (advance mean for next re-linearization)
12:     $\boldsymbol{\eta}^{(i)} \leftarrow \boldsymbol{\eta}^{(i)} + \varepsilon_j(\mathbf{A}\boldsymbol{\eta}^{(i)} + \mathbf{b})$  for all  $i$ 
13:  end for
14:  Update (EKF/UKF): same as Algorithm 7 (kernel only; not reported)
15:  Output:  $m_k = \frac{1}{N} \sum_i \boldsymbol{\eta}^{(i)}$ ,  $P_k = \frac{1}{N} \sum_i (\boldsymbol{\eta}^{(i)} - m_k)(\boldsymbol{\eta}^{(i)} - m_k)^\top$ 
16: end for

```

Algorithm 9 LEDH Flow Filter (Local Per-Particle Linearization)

Require: Same setup as Algorithm 7

```

1: for  $k = 1, 2, \dots, T$  do
2:   Feedback, Predict (EKF/UKF), Predict (particles): same as Algorithm 7
3:    $\lambda \leftarrow 0$ 
4:   for  $j = 1, 2, \dots, n_\lambda$  do
5:      $\lambda \leftarrow \lambda + \varepsilon_j$ 
6:     for each particle  $i$  (batched) do
7:        $\mathbf{H}^{(i)} \leftarrow \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \Big|_{\boldsymbol{\eta}^{(i)}}$  (re-linearize at each particle's own position)
8:        $\mathbf{e}^{(i)} \leftarrow \mathbf{h}(\boldsymbol{\eta}^{(i)}) - \mathbf{H}^{(i)}\boldsymbol{\eta}^{(i)}$  (nonlinear offset correction)
9:        $\mathbf{A}^{(i)} \leftarrow -\frac{1}{2}\mathbf{P}\mathbf{H}^{(i)\top}(\lambda\mathbf{H}^{(i)}\mathbf{P}\mathbf{H}^{(i)\top} + \mathbf{R})^{-1}\mathbf{H}^{(i)}$ 
10:       $\mathbf{b}^{(i)} \leftarrow (\mathbf{I} + 2\lambda\mathbf{A}^{(i)})[(\mathbf{I} + \lambda\mathbf{A}^{(i)})\mathbf{P}\mathbf{H}^{(i)\top}\mathbf{R}^{-1}(\mathbf{z}_k - \mathbf{e}^{(i)}) + \mathbf{A}^{(i)}\bar{\boldsymbol{\eta}}_0]$ 
11:       $\boldsymbol{\eta}^{(i)} \leftarrow \boldsymbol{\eta}^{(i)} + \varepsilon_j(\mathbf{A}^{(i)}\boldsymbol{\eta}^{(i)} + \mathbf{b}^{(i)})$ 
12:    end for
13:  end for
14:  Update (EKF/UKF): same as Algorithm 7 (kernel only; not reported)
15:  Output:  $m_k = \frac{1}{N} \sum_i \boldsymbol{\eta}^{(i)}$ ,  $P_k = \frac{1}{N} \sum_i (\boldsymbol{\eta}^{(i)} - m_k)(\boldsymbol{\eta}^{(i)} - m_k)^\top$ 
16: end for

```

4.3.5 Experiments

4.4 Invertible Flow Filter

4.4.1 Why do we need Jacobian computation?

The invertible flow filter [4] corrects for the volume distortion introduced by discretization. The continuous flow $d\mathbf{x}/d\lambda = f(\mathbf{x}, \lambda)$ is exact by construction: the Fokker-Planck equation guarantees that following the flow moves probability mass correctly from $g(x)$ to $g(x)h(x)$. In practice we integrate with finite steps ε_j , so each λ step applies the discrete affine map

$$\boldsymbol{\eta}^{(i)} \leftarrow (I + \varepsilon_j \mathbf{A}_j^{(i)}) \boldsymbol{\eta}^{(i)} + \varepsilon_j \mathbf{b}_j^{(i)}. \quad (67)$$

This is an invertible linear map with Jacobian $\det(I + \varepsilon_j \mathbf{A}_j^{(i)})$. Unless $\det(I + \varepsilon_j \mathbf{A}) = 1$ for every step (i.e., the map is volume-preserving), the discrete flow changes the volume element and therefore distorts the density. The continuous flow does not have this problem—the exact Fokker-Planck solution implicitly accounts for volume changes—but any finite discretization introduces this error. To correct for it, we

re-introduce importance weights. The weight accumulated over all n_λ steps of the flow at time k is

$$\log w_k^{(i)} = \log w_{k-1}^{(i)} + \sum_{j=1}^{n_\lambda} \log |\det(I + \varepsilon_j A_j^{(i)})|, \quad (68)$$

which is then normalized across particles. A positive log-Jacobian means the map expanded the local volume (particles spread out), contributing upward weight; a negative one means compression, contributing downward weight. After the full flow the particle set $\{\boldsymbol{\eta}^{(i)}, w^{(i)}\}$ is a weighted approximation to the posterior, and the usual ESS-triggered resampling is applied.

For the EDH variants (both fixed and intermediate re-linearization), A_j is a global matrix—the same for every particle at step j . The Jacobian factor $\det(I + \varepsilon_j A_j)$ is therefore identical for all particles. Under weight normalization, a common multiplicative factor in all $w^{(i)}$ cancels exactly. The relative weights remain uniform, and no correction is needed—the EDH flow filter without weights is already self-consistent. This is why the unweighted EDH flow (Algorithm 7 and 8) is not simply an approximation that ignores the Jacobian; the Jacobian genuinely does not affect the normalized weights.

For LEDH, $A_j^{(i)}$ is particle-specific: different particles are linearized at different positions, so they are stretched or compressed by different amounts at each step. The Jacobian $\det(I + \varepsilon_j A_j^{(i)})$ now varies across particles and does not cancel under normalization. Ignoring it would let the flow silently distort the distribution in a particle-dependent way. The invertible correction in (68) accounts for exactly this: it reweights each particle by how much the flow expanded or contracted the space around it, restoring consistency between the transported point cloud and the target density.

At each λ step, $A^{(i)} = -\frac{1}{2} \mathbf{P} \mathbf{H}^{(i)\top} (\lambda \mathbf{H}^{(i)} \mathbf{P} \mathbf{H}^{(i)\top} + \mathbf{R})^{-1} \mathbf{H}^{(i)}$ has rank at most d_z (the observation dimension), because it is a product $U W V^\top$ where $U = \mathbf{P} \mathbf{H}^{(i)\top}$ is $d_x \times d_z$. A naive determinant over $d_x \times d_x$ matrices would be expensive. Instead, apply Sylvester’s determinant theorem $\det(I_{d_x} + U V^\top) = \det(I_{d_z} + V^\top U)$ to reduce the computation to a $d_z \times d_z$ determinant:

$$\log |\det(I_{d_x} + \varepsilon_j A^{(i)})| = \log |\det(I_{d_z} - \frac{\varepsilon_j}{2} \mathbf{H}^{(i)} \mathbf{P} \mathbf{H}^{(i)\top} (\lambda \mathbf{H}^{(i)} \mathbf{P} \mathbf{H}^{(i)\top} + \mathbf{R})^{-1})|. \quad (69)$$

When $d_z \ll d_x$ this is a substantial saving. In the common case $d_z = d_x = 1$ it reduces to a scalar logarithm. The accumulated log-weight over all n_λ steps is computed by summing this scalar (or small-matrix determinant) at each step, adding negligible overhead compared to the flow integration itself.

Algorithm 10 EDH Invertible Particle Flow Filter (Intermediate Re-linearization)

Require: Particles $\{\boldsymbol{\eta}^{(i)}, w^{(i)}\}_{i=1}^N$, parallel EKF/UKF, ESS threshold

```
1: for  $k = 1, 2, \dots, T$  do
2:   Predict (EKF/UKF):  $(\bar{\boldsymbol{\eta}}_0, \mathbf{P}) \leftarrow$  EKF/UKF predict
3:   Predict (particles):  $\boldsymbol{\eta}_0^{(i)} \sim p(\mathbf{x}_k \mid \boldsymbol{\eta}^{(i)})$ , store pre-flow particles
4:   Precompute  $\mathbf{R}^{-1}$ 
5:   Initialize  $\lambda \leftarrow 0$ ,  $\boldsymbol{\eta}_1^{(i)} \leftarrow \boldsymbol{\eta}_0^{(i)}$ ,  $\bar{\boldsymbol{\eta}}^{(\lambda)} \leftarrow \bar{\boldsymbol{\eta}}_0$ 
6:   for  $j = 1, 2, \dots, n_\lambda$  do
7:      $\lambda \leftarrow \lambda + \varepsilon_j$ 
8:      $\mathbf{H} \leftarrow \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\bar{\boldsymbol{\eta}}^{(\lambda)}}$  (re-linearize at current global mean)
9:      $\mathbf{e} \leftarrow \mathbf{h}(\bar{\boldsymbol{\eta}}^{(\lambda)}) - \mathbf{H}\bar{\boldsymbol{\eta}}^{(\lambda)}$  (nonlinear correction)
10:     $\mathbf{A} \leftarrow -\frac{1}{2}\mathbf{P}\mathbf{H}^\top(\lambda\mathbf{H}\mathbf{P}\mathbf{H}^\top + \mathbf{R})^{-1}\mathbf{H}$ 
11:     $\mathbf{b} \leftarrow (\mathbf{I} + 2\lambda\mathbf{A})[(\mathbf{I} + \lambda\mathbf{A})\mathbf{P}\mathbf{H}^\top\mathbf{R}^{-1}(\mathbf{z}_k - \mathbf{e}) + \mathbf{A}\bar{\boldsymbol{\eta}}_0]$ 
12:     $\bar{\boldsymbol{\eta}}^{(\lambda)} \leftarrow \bar{\boldsymbol{\eta}}^{(\lambda)} + \varepsilon_j(\mathbf{A}\bar{\boldsymbol{\eta}}^{(\lambda)} + \mathbf{b})$  (advance mean for next step)
13:     $\boldsymbol{\eta}_1^{(i)} \leftarrow \boldsymbol{\eta}_1^{(i)} + \varepsilon_j(\mathbf{A}\boldsymbol{\eta}_1^{(i)} + \mathbf{b})$  for all  $i$ 
14:  end for
15:  Weight update:
16:   $\tilde{w}^{(i)} \leftarrow w^{(i)} \cdot \frac{p(\mathbf{z}_k \mid \boldsymbol{\eta}_1^{(i)})}{p(\mathbf{z}_k \mid \boldsymbol{\eta}_0^{(i)})}$  (importance weight correction)
17:   $w^{(i)} \leftarrow \tilde{w}^{(i)} / \sum_j \tilde{w}^{(j)}$  (normalize)
18:   $\boldsymbol{\eta}^{(i)} \leftarrow \boldsymbol{\eta}_1^{(i)}$ 
19:  if  $N_{\text{eff}} = 1 / \sum_i (w^{(i)})^2 < \text{threshold}$  then
20:    Resample  $\{\boldsymbol{\eta}^{(i)}, w^{(i)}\}$  (systematic resampling)
21:  end if
22:  Update (EKF/UKF):  $(\bar{\boldsymbol{\eta}}, \mathbf{P}) \leftarrow$  EKF/UKF update( $\bar{\boldsymbol{\eta}}_0, \mathbf{P}, \mathbf{z}_k$ )
23: end for
```

Algorithm 11 LEDH Invertible Particle Flow Filter (Local Linearization + Jacobian Tracking)

Require: Particles $\{\boldsymbol{\eta}^{(i)}, w^{(i)}\}_{i=1}^N$, per-particle batched EKF/UKF, ESS threshold

Require: Weight clip range C (optional, e.g. $C = 50$)

```

1: for  $k = 1, 2, \dots, T$  do
2:   Predict (batched EKF/UKF):
3:    $(\bar{\boldsymbol{\eta}}_0^{(i)}, \mathbf{P}^{(i)}) \leftarrow$  batched EKF/UKF predict( $\boldsymbol{\eta}^{(i)}, \mathbf{P}^{(i)}$ ) for all  $i$ 
4:   Predict (particles):  $\boldsymbol{\eta}_0^{(i)} \sim p(\mathbf{x}_k | \boldsymbol{\eta}^{(i)})$ 
5:   Precompute  $\mathbf{R}^{-1}$ 
6:   Initialize  $\lambda \leftarrow 0$ ,  $\boldsymbol{\eta}_1^{(i)} \leftarrow \boldsymbol{\eta}_0^{(i)}$ ,  $\log \theta^{(i)} \leftarrow 0$ 
7:   for  $j = 1, 2, \dots, n_\lambda$  do
8:      $\lambda \leftarrow \lambda + \varepsilon_j$ 
9:     for each particle  $i$  (batched) do
10:       $\mathbf{H}^{(i)} \leftarrow \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \Big|_{\boldsymbol{\eta}_1^{(i)}} \quad \text{(re-linearize at current position)}$ 
11:       $\mathbf{e}^{(i)} \leftarrow \mathbf{h}(\boldsymbol{\eta}_1^{(i)}) - \mathbf{H}^{(i)} \boldsymbol{\eta}_1^{(i)}$ 
12:      Compute  $\mathbf{A}^{(i)}(\lambda), \mathbf{b}^{(i)}(\lambda)$  using  $\mathbf{H}^{(i)}, \mathbf{P}^{(i)}, \bar{\boldsymbol{\eta}}_0^{(i)}$ 
13:      end for
14:       $\boldsymbol{\eta}_1^{(i)} \leftarrow \boldsymbol{\eta}_1^{(i)} + \varepsilon_j (\mathbf{A}^{(i)} \boldsymbol{\eta}_1^{(i)} + \mathbf{b}^{(i)})$  for all  $i$ 
15:       $\mathbf{M}^{(i)} \leftarrow \mathbf{I} + \varepsilon_j \mathbf{A}^{(i)} \quad \text{(local Jacobian of flow map)}$ 
16:       $\log \theta^{(i)} \leftarrow \log \theta^{(i)} + \log |\det(\mathbf{M}^{(i)})| \quad \text{(accumulate log-det)}$ 
17:    end for
18:    Weight update with Jacobian correction:
19:     $\log \theta^{(i)} \leftarrow \log \theta^{(i)} - \max_j \log \theta^{(j)} \quad \text{(stabilize before exp)}$ 
20:     $\theta^{(i)} \leftarrow \exp(\log \theta^{(i)})$ 
21:     $\tilde{w}^{(i)} \leftarrow w^{(i)} \cdot \frac{p(\mathbf{z}_k | \boldsymbol{\eta}_1^{(i)})}{p(\mathbf{z}_k | \boldsymbol{\eta}_0^{(i)})} \cdot \theta^{(i)}$ 
22:    if clip range  $C$  specified then
23:       $\log \tilde{w}^{(i)} \leftarrow \text{clip}(\log \tilde{w}^{(i)}, -C, +C) \quad \text{(prevent overflow)}$ 
24:    end if
25:     $w^{(i)} \leftarrow \tilde{w}^{(i)} / \sum_j \tilde{w}^{(j)}$ 
26:     $\boldsymbol{\eta}^{(i)} \leftarrow \boldsymbol{\eta}_1^{(i)}$ 
27:    Update (batched EKF/UKF): update per-particle covariances
28:    if  $N_{\text{eff}} < \text{threshold}$  then
29:      Resample  $\{\boldsymbol{\eta}^{(i)}, w^{(i)}\}$ 
30:    end if
31:  end for

```

4.5 Stochastic Flow Filter

The stochastic particle flow filter of [5] generalises the deterministic EDH flow from an ODE to a stochastic differential equation:

$$d\mathbf{x} = [A(\lambda)\mathbf{x} + b(\lambda)] d\lambda + \sqrt{Q} dW_\lambda,$$

where $Q = \text{diag}(q_1, \dots, q_d)$ is a diagonal diffusion matrix. The drift includes a score correction (Corollary 2.1 of [5]) so that the posterior is exact in the continuous-time limit for any choice of Q .

The deterministic EDH flow (ODE) can freely re-linearise H at each particle's position—the local (LEDH) approach—because the ODE drift is simply a velocity field that each particle follows independently. The stochastic flow is fundamentally different. The drift contains a score correction:

$$f_{\text{stoch}}(\mathbf{x}, \lambda) = f_{\text{det}}(\mathbf{x}, \lambda) + \frac{1}{2} Q \nabla_{\mathbf{x}} \log p(\mathbf{x}, \lambda),$$

where $p(\mathbf{x}, \lambda) \propto g(\mathbf{x}) h(\mathbf{x})^\lambda$ is the homotopy density. This correction compensates for the injected diffusion noise so that the particle density stays on the homotopy path. Formally, the drift must satisfy the Fokker–Planck equation

$$\frac{\partial p}{\partial \lambda} = -\nabla \cdot (f p) + \frac{1}{2} \nabla \cdot (Q Q^\top \nabla p),$$

a *global* constraint: there must exist a single density $p(\mathbf{x}, \lambda)$ whose score $\nabla \log p$ appears in the drift for every particle simultaneously.

Under global linearization (H fixed at $\bar{\eta}_0$), $p(\mathbf{x}, \lambda)$ is Gaussian with precision $M(\lambda) = P_0^{-1} + \lambda H^\top R^{-1} H$, and the score

$$\nabla \log p(\mathbf{x}, \lambda) = -M(\lambda)(\mathbf{x} - m(\lambda))$$

is a single linear function of $\mathbf{x}$ shared by all particles—consistent with the Fokker–Planck equation.

If one were to re-linearise at each particle $\mathbf{x}_i$ (as LEDH does for ODE flows), particle i would use its own H_i , giving it its own precision $M_i(\lambda)$, mean $m_i(\lambda)$, and score $-M_i(\mathbf{x} - m_i)$. No single density has all of these as its score simultaneously—the Fokker–Planck equation cannot be satisfied. The particles would no longer be samples from a coherent density, and the theoretical guarantee that the flow preserves the homotopy path breaks down. This is why stochastic particle flows in the literature universally use global linearization, while the local (LEDH) variant is restricted to deterministic flows. Given this fact, in the code base, we can directly inherit the structure of EDH with small override to the flow parameter. For strong non-linear model, we need to add local corrections to the flow parameters, or re-linearize at the intermediate time. But for Stochastic flow, re-linearize is not possible, we cannot update the observation coefficient, but it turns out adding local correction will help stabilize the flow.

The drift parameters $A(\lambda)$ and $b(\lambda)$ are derived under the assumption of a linear observation model $h(\mathbf{x}) = H\mathbf{x}$. In the pure global formulation—where H is computed once at the prior mean $\bar{\eta}_0$ and the observation z is used directly—this linear model is self-consistent and no additional correction is needed.

However, in the re-linearisation framework (as in LEDH), one expands $h(\mathbf{x})$ at an intermediate point η during the flow. The first-order Taylor expansion

$$h(\mathbf{x}) \approx h(\eta) + H(\eta)(\mathbf{x} - \eta) = H(\eta)\mathbf{x} + \underbrace{[h(\eta) - H(\eta)\eta]}_e,$$

naturally produces a zeroth-order residual $e = h(\eta) - H(\eta)\eta$ that accounts for the nonlinearity. In our local correction variant, we borrow this idea for the global formulation: compute $e_0 = h(\bar{\eta}_0) - H\bar{\eta}_0$ once, then replace z with $z_{\text{corr}} = z - e_0$ in both the deterministic drift and the score correction. For linear observations $e_0 = 0$ and the two formulations coincide.

Empirically, this correction is essential for nonlinear models. On the range-bearing model, the uncorrected stochastic EDH diverges, while adding the local correction makes the filter work (Figures ?? and ??). The correction costs only one additional evaluation of $h(\bar{\eta}_0)$ per time step. As for stiffness mitigation, the drift matrix $A(\lambda)$ has eigenvalues that are most extreme near $\lambda = 0$, where the precision $M(\lambda) = P_0^{-1} + \lambda H^\top R^{-1} H$ is dominated by the anisotropic prior P_0^{-1} . As $\lambda \rightarrow 1$, measurement information regularizes M and the stiffness diminishes. With a sufficiently fine uniform grid (small $d\lambda$), the Euler–Maruyama integrator stays within the stability region even near $\lambda = 0$ —a fine uniform schedule is not stressed by the stiff system.

In practice, one can do better with a geometric (exponential) schedule: step sizes $\varepsilon_j = \varepsilon_1 q^{j-1}$ with ratio $q = 1.2$, which concentrates steps near $\lambda = 0$ where they are most needed and uses larger steps near $\lambda = 1$ where the system is well-conditioned. This simple engineering choice already resolves the practical stiffness issue without solving any optimization problem. I spent sometime to test and reproduce the result of the stiffness mitigation paper but could not reproduce. The filter itself works fine, but I find the usefulness of the method proposed by the author is limited, and the stiffness in the example can be mitigated by fine uniform step size. One thing I did not try is to switch the integration scheme. We evaluate our parameter after we update the step counts, follow the pseudocode from [6]. As we know the first step has strong stiffness problem, this could be the reason why I could not reproduce the paper.

4.6 Kernel Flow Filter

4.6.1 Overview

kernel-embedded particle flow filter in an RK:[7, 8]: Instead of moving particles with certain schedule, we move the particle along the trajectory where the difference between the particle distribution with the target is decreasing as fast as possible. Define target posterior $\pi(x) := p(x_t|z_{1:t})$ and intermediate density $q_\lambda(x)$ at pseudo-time $\lambda \in [0, 1]$. With $q_0 = p(x_t|z_{1:t-1})$ (prior), $q_1 = \pi$ (posterior).

$$\frac{dx_\lambda}{d\lambda} = v_\lambda(x_\lambda) \quad (70)$$

Choose velocity v_λ to minimize KL divergence as fast as possible:

$$D_{KL}(q_\lambda \parallel \pi) = \int q_\lambda(x) \log \frac{q_\lambda(x)}{\pi(x)} dx \quad (71)$$

we need to formulate the above equation into an optimization problem. But the intuition of this method is clear, you want to evolve the point along a "straight" direction to the desire point. On the implementation side.

4.6.2 Derivation

Our problem is the same as particle filter particle flow, move the particles to the desired location, without touching the weights, so that we get the target probability distribution $p(x_t|y_{1:t}) \propto p(y_t|x_t)p(x_t|y_{1:t-1})$.

Optimal Transport via Gradient Flow Define target posterior $\pi(x) := p(x_t|y_{1:t})$ and intermediate density $q_\lambda(x)$ at pseudo-time $\lambda \in [0, 1]$. With $q_0 = p(x_t|y_{1:t-1})$ (prior), $q_1 = \pi$ (posterior).

$$\frac{dx_\lambda}{d\lambda} = v_\lambda(x_\lambda) \quad (72)$$

Choose velocity v_λ to minimize KL divergence as fast as possible:

$$D_{KL}(q_\lambda \parallel \pi) = \int q_\lambda(x) \log \frac{q_\lambda(x)}{\pi(x)} dx \quad (73)$$

we need to formulate the above equation into an optimization problem. But the intuition of this method is clear, you want to evolve the point along a "straight" direction to the desire point.

Deriving Optimal Velocity via Kernel Restriction Under velocity field v , density evolves via Liouville equation:

$$\frac{\partial q_\lambda}{\partial \lambda} = -\nabla_x \cdot (q_\lambda v) \quad (74)$$

Time derivative of D_{KL} :

$$\frac{d}{d\lambda} D_{KL} = \int \frac{\partial q_\lambda}{\partial \lambda} [\log q_\lambda(x) - \log \pi(x) + 1] dx \quad (75)$$

$$= - \int \nabla_x \cdot (q_\lambda v) [\log q_\lambda(x) - \log \pi(x) + 1] dx \quad (76)$$

Using $\nabla_x q_\lambda = q_\lambda \nabla_x \log q_\lambda$ and integrating by parts:

$$\frac{d}{d\lambda} D_{KL} = - \int q_\lambda [\nabla_x \log \pi(x) \cdot v + \nabla_x \cdot v] dx \quad (77)$$

The derivative with respect to the evolution parameter gives the vector field along the trajectory. To make things computatble, we restrict v to functions generated by kernel $K(x, x')$. Any such function has form:

$$v(x) = \int \alpha(x') K(x', x) dx' \quad (78)$$

for some coefficient function $\alpha(x')$. Substitute this into directional derivative.

$$\frac{d}{d\lambda} D_{KL} = - \int q_\lambda(x) \left[\nabla_x \log \pi(x) \cdot \int \alpha(x') K(x', x) dx' + \nabla_x \cdot \int \alpha(x') K(x', x) dx' \right] dx \quad (79)$$

Interchange integrals and use integration by parts:

$$\frac{d}{d\lambda} D_{KL} = \int \alpha(x) \cdot \phi(x) dx \quad (80)$$

where

$$\phi(x) = - \int q_\lambda(x') [K(x, x') \nabla_{x'} \log \pi(x') + \nabla_{x'} K(x, x')] dx' \quad (81)$$

We have $\frac{d}{d\lambda} D_{KL} = \langle \alpha, \phi \rangle$ (inner product in function space).

To minimize this quantity (make KL decrease fastest), choose α antiparallel to ϕ . By Cauchy-Schwarz, the minimum is achieved when:

$$\alpha(x) = -\phi(x) \quad (82)$$

Therefore, optimal velocity field:

$$v_\lambda(x) = \int \alpha(x') K(x', x) dx' \quad (83)$$

$$= - \int \phi(x') K(x', x) dx' \quad (84)$$

By the reproducing property of the kernel, this simplifies to:

$$\boxed{v_\lambda(x) = \mathbb{E}_{x' \sim q_\lambda} [K(x, x') \nabla_{x'} \log \pi(x') + \nabla_{x'} K(x, x')]} \quad (85)$$

Monte Carlo approximation with particles $\{x_\lambda^{(i)}\}_{i=1}^{N_p} \sim q_\lambda$:

$$v_\lambda(x) \approx \frac{1}{N_p} \sum_{i=1}^{N_p} [K(x_\lambda^{(i)}, x) \nabla \log \pi(x_\lambda^{(i)}) + \nabla K(x_\lambda^{(i)}, x)] \quad (86)$$

Posterior gradient:

$$\nabla \log \pi(x) = \nabla \log p(y_t | x) + \nabla \log p(x | y_{1:t-1}) \quad (87)$$

First term = likelihood gradient (known). Second term = prior gradient (approximated from ensemble).

Update rule:

$$x_{\lambda+\epsilon}^{(j)} = x_\lambda^{(j)} + \frac{\epsilon}{N_p} \sum_{i=1}^{N_p} [K(x_\lambda^{(i)}, x_\lambda^{(j)}) \nabla \log \pi(x_\lambda^{(i)}) + \nabla K(x_\lambda^{(i)}, x_\lambda^{(j)})] \quad (88)$$

All particles maintain equal weight $w^{(i)} = 1/N_p$ throughout. No resampling needed.

Generality: Works for any differentiable likelihood $p(y_t | x_t)$ - not limited to Gaussian noise.

Scalar Kernel Standard choice: isotropic Gaussian

$$K(x, x') = \exp \left(-\frac{1}{2\alpha\sigma^2} \|x - x'\|^2 \right), \quad \alpha \sim n_x \quad (89)$$

Gradient: $\nabla_x K(x, x') = -\frac{1}{\alpha\sigma^2} (x - x') K(x, x')$

Single scalar $K(x, x')$ weights all components identically.

Matrix-Valued Kernel Measure distance independently per component:

$$\mathbf{K}(x, x') = \text{diag} [K^{(1)}(x, x'), \dots, K^{(n_x)}(x, x')] \quad (90)$$

where

$$K^{(d)}(x, x') = \exp \left(-\frac{(x^{(d)} - x'^{(d)})^2}{2\alpha(\sigma^{(d)})^2} \right), \quad \alpha = \frac{1}{N_p} \quad (91)$$

Component-wise repulsion:

$$[\nabla_x \cdot \mathbf{K}(x, x')]^{(d)} = -\frac{x^{(d)} - x'^{(d)}}{\alpha(\sigma^{(d)})^2} K^{(d)}(x, x') \quad (92)$$

Observed dimensions maintain diversity regardless of unobserved dimension spread.

4.6.3 Matrix Kernel Preventing Collapse

4.6.4 Algorithm

Algorithm 12 Kernel Flow Filter — Scalar Kernel

Require: Particles $\{\mathbf{x}^{(i)}\}_{i=1}^N$ with equal weights, scalar bandwidth $\alpha = 1/N$

Require: Localization matrix $\mathbf{C}$ (Schur product mask), pseudo-time steps n_s , step size Δt

```

1: for  $k = 1, 2, \dots, T$  do
2:   Predict:  $\mathbf{x}_0^{(i)} \sim p(\mathbf{x}_k | \mathbf{x}^{(i)})$  for all  $i$ 
3:   Compute ensemble mean  $\bar{\mathbf{x}}_0$ , localized covariance  $\mathbf{B} = \mathbf{C} \circ \hat{\mathbf{P}}_{\text{sample}}$ 
4:   Precompute  $\mathbf{B}^{-1}$ ,  $\mathbf{R}^{-1}$ , bandwidth  $\sigma^2 = \alpha \cdot \text{tr}(\mathbf{B})/d$ 
5:   for  $s = 1, \dots, n_s$  do
6:     for each particle  $j$  (batched) do
7:       Posterior gradient:
8:        $\nabla \log p(\mathbf{z}_k | \mathbf{x}^{(j)}) = \mathbf{H}^{(j)\top} \mathbf{R}^{-1}(\mathbf{z}_k - \mathbf{h}(\mathbf{x}^{(j)}))$ 
9:        $\nabla \log p(\mathbf{x}^{(j)}) = -\mathbf{B}^{-1}(\mathbf{x}^{(j)} - \bar{\mathbf{x}}_0)$ 
10:       $\nabla \log \pi(\mathbf{x}^{(j)}) = \nabla \log p(\mathbf{z}_k | \mathbf{x}^{(j)}) + \nabla \log p(\mathbf{x}^{(j)})$ 
11:      Scalar kernel:  $K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp\left(-\frac{\|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2}{2\sigma^2}\right)$ 
12:      Kernel gradient:  $\nabla_{\mathbf{x}^{(i)}} K = -\frac{1}{\sigma^2}(\mathbf{x}^{(i)} - \mathbf{x}^{(j)})K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$ 
13:      Flow:  $\mathbf{v}^{(j)} = \frac{1}{N} \sum_{i=1}^N [K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \nabla \log \pi(\mathbf{x}^{(i)}) + \nabla_{\mathbf{x}^{(i)}} K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})]$ 
14:    end for
15:    Precondition:  $\mathbf{v}^{(j)} \leftarrow \mathbf{B} \mathbf{v}^{(j)}$ 
16:     $\mathbf{x}^{(j)} \leftarrow \mathbf{x}^{(j)} + \Delta t \cdot \mathbf{v}^{(j)}$  for all  $j$ 
17:  end for
18: end for

```

Algorithm 13 Kernel Flow Filter — Matrix (Tensor) Kernel

Require: Same setup as scalar kernel. Per-dimension bandwidth: $(\sigma^{(d)})^2 = \alpha \cdot B_{dd}$

```

1: for  $k = 1, 2, \dots, T$  do
2:   Predict and compute  $\bar{\mathbf{x}}_0, \mathbf{B}, \mathbf{B}^{-1}, \mathbf{R}^{-1}$  as in scalar kernel
3:   for  $s = 1, \dots, n_s$  do
4:     for each particle  $j$  (batched) do
5:       Compute  $\nabla \log \pi(\mathbf{x}^{(j)})$  as before
6:       Matrix kernel (diagonal, per-dimension):
7:        $K^{(d)}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp\left(-\frac{(x_d^{(i)} - x_d^{(j)})^2}{2\alpha(\sigma^{(d)})^2}\right)$  for each dimension  $d$ 
8:        $[\nabla_{\mathbf{x}^{(i)}} \cdot \mathbf{K}]^{(d)} = -\frac{x_d^{(i)} - x_d^{(j)}}{\alpha(\sigma^{(d)})^2} K^{(d)}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$ 
9:       Per-dimension flow:
10:       $v_d^{(j)} = \frac{1}{N} \sum_{i=1}^N [K^{(d)}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) [\nabla \log \pi(\mathbf{x}^{(i)})]_d + [\nabla_{\mathbf{x}^{(i)}} \cdot \mathbf{K}]^{(d)}]$ 
11:    end for
12:    Precondition:  $\mathbf{v}^{(j)} \leftarrow \mathbf{B} \mathbf{v}^{(j)}$ 
13:     $\mathbf{x}^{(j)} \leftarrow \mathbf{x}^{(j)} + \Delta t \cdot \mathbf{v}^{(j)}$  for all  $j$ 
14:  end for
15: end for

```

4.7 Filter Comparison Summary

Overall, there is no one-fits-all filter option. When the model is non-linear but with additive noise, then flow based approach would work with carefully tuned hyper parameters. As you can see, there is a lot of freedom to adjust the flow equation implementation. With multiplicative noise, I cannot say for certain, but I would expect most of the expansion approach would fail because the observation is not useful at all, for flow evolution.

5 Resampling and Optimal Transport

5.1 Two Problems with Resampling

Previously, we have discussed the sample impoverishment problem introduced by low quality resampling. Therefore, we need to think about how to improve it. Before and after resampling, we are preserving the distribution. In [9], the author proposed to use optimal transport to do the resampling. An easy way to see this is that, before the resampling, we have $\{\omega_i, x_i\}$, and after the resampling, we have $\{\tilde{\omega}_i, \tilde{x}_i\}$. If we think x_i are basis vectors and ω_i are the coefficients, we are dealing with a linear transformation problem. The transformation constructs an equal-weight approximation that reduces resampling variance and better preserves the structure of the distribution. To put it sharply, we want to make sure $\{\omega_i, x_i\}$ and $\{\tilde{\omega}_i, \tilde{x}_i\}$ represent the same distribution and $\tilde{\omega}_i \equiv \frac{1}{N}$, where N is the total number of particles.

There are two pressures on resampling that we need to address separately:

Problem 1: Sampling noise. OT resampling is deterministic instead of stochastic, which means it avoids the sampling noise introduced by systematic resampling, therefore removing its contribution to the variance of the particle cloud, namely increasing the quality of covariance estimation. But the classical OT formulation relies on solving a discrete Linear Programming (LP) problem to find the optimal transport plan. LP solvers like the Simplex algorithm or Interior Point methods walk along edges of the polytope or navigate through its interior. They do not need derivatives of the optimal solution with respect to inputs—they only need the cost matrix (particle distances) and constraints (matching weights). This is where the second problem comes in.

Problem 2: Non-differentiability. The LP-based OT plan is not differentiable with respect to the inputs. For our purpose—building a differentiable particle filter whose log-likelihood can be differentiated with respect to model parameters—this is a showstopper. The key innovation of [10] is to introduce entropy regularization so that the optimal coupling P^* becomes differentiable as a function of the cost matrix, therefore turning the whole resampling operation differentiable in principle. The regularized transport cost is

$$W_{2,\epsilon}^2(\alpha_N, \beta_N) = \min_{P \in S(a,b)} \sum_{i,j=1}^N p_{i,j} \left(c_{i,j} + \epsilon \log \frac{p_{i,j}}{a_i b_j} \right) \quad (93)$$

The practical issue is computing the gradient faithfully through the iterative Sinkhorn solver that finds P^* . We will see in the following subsections that the regularized OT map is differentiable in principle, but getting an accurate backward pass requires care. This is our focus for the rest of this section.

By adding the entropy term $\epsilon \log \frac{p_{i,j}}{a_i b_j}$ instead of $\epsilon \log p_{i,j}$, the authors of [10] made it possible to recover the transport plan from the dual variables alone. Note that this objective is equivalent (up to constants under fixed marginals) to the standard entropy-regularized form $\langle \mathbf{C}, \mathbf{P} \rangle - \epsilon \mathbf{H}(\mathbf{P})$ that we will use in the Sinkhorn derivation below. The dual recovery step is $\mathcal{O}(N)$ per entry, compared to $\mathcal{O}(N^2)$ for the primal LP of [9], which opens the possibility for modeling more complicated systems with large numbers of particles.

5.2 Soft Resampling

Soft resampling [22] is a compromise between fully stochastic systematic resampling and fully deterministic OT resampling. The idea is to blend the particle weights with a uniform distribution before selection, then apply an importance correction after. The algorithm is given below.

Algorithm 14 Soft Resampling

Require: Particles $\{\mathbf{x}^{(i)}, w^{(i)}\}_{i=1}^N$ (weights normalized: $\sum_i w^{(i)} = 1$)

Require: Trade-off parameter $\alpha \in [0, 1]$ ($\alpha = 1$: hard resampling, $\alpha = 0$: pure importance sampling)

1: **Mixture distribution:**

2: $q^{(i)} \leftarrow \alpha w^{(i)} + (1 - \alpha)/N$ for $i = 1, \dots, N$ (q is strictly positive)

3: **Systematic sampling from q :**

4: Draw $u \sim \mathcal{U}[0, 1/N]$

5: $u^{(i)} \leftarrow u + (i - 1)/N$ for $i = 1, \dots, N$

6: Select indices j_i via cumulative sum of q : $\sum_{m=1}^{j_i-1} q^{(m)} < u^{(i)} \leq \sum_{m=1}^{j_i} q^{(m)}$

7: $\tilde{\mathbf{x}}^{(i)} \leftarrow \mathbf{x}^{(j_i)}$ for $i = 1, \dots, N$

8: **Importance weight correction:**

9: $\tilde{w}^{(i)} \leftarrow w^{(j_i)}/q^{(j_i)}$ for $i = 1, \dots, N$

10: $\tilde{w}^{(i)} \leftarrow \tilde{w}^{(i)} / \sum_j \tilde{w}^{(j)}$ (normalize)

11: **return** $\{\tilde{\mathbf{x}}^{(i)}, \tilde{w}^{(i)}\}_{i=1}^N$

The critical observation is that soft resampling is only *partially* differentiable. The selection step (line 6) is still discrete—the cumulative-sum lookup picks an integer index j_i , and the gradient does not flow through that choice. Only the importance weight correction $w^{(j_i)}/q^{(j_i)}$ on line 9 is differentiable through w . The selection gradient—how θ affects *which* particle gets picked—is still zero. We will make this precise in Section 5.4.

5.3 Entropy-Regularized OT (Sinkhorn) Resampling

The entropy-regularized OT resampling problem is a direct application of the Monge-Kantorovich framework. To set up the problem, recall that in the Monge-Kantorovich problem we have two probability vectors $\mathbf{a} \in \mathbb{R}_+^N$ and $\mathbf{b} \in \mathbb{R}_+^N$ and a cost matrix $\mathbf{C} \in \mathbb{R}^{N \times N}$. The goal is to find the optimal coupling matrix $\mathbf{P} \in \mathbb{R}_+^{N \times N}$, where P_{ij} describes the flow of mass from bin i to bin j , minimizing total transport cost:

$$\min_{\mathbf{P} \in \mathcal{U}(\mathbf{a}, \mathbf{b})} \langle \mathbf{C}, \mathbf{P} \rangle, \quad \mathcal{U}(\mathbf{a}, \mathbf{b}) = \{\mathbf{P} \in \mathbb{R}_+^{N \times N} : \mathbf{P} \mathbf{1}_N = \mathbf{a}, \mathbf{P}^\top \mathbf{1}_N = \mathbf{b}\}. \quad (94)$$

In the resampling context, we set $\mathbf{a} = \mathbf{w}$ (particle weights as the source marginal), $\mathbf{b} = \frac{1}{N} \mathbf{1}_N$ (the uniform distribution as the target marginal), and $C_{ij} = \|\tilde{\mathbf{x}}^{(i)} - \tilde{\mathbf{x}}^{(j)}\|^2/2$ as the squared-distance cost between normalized particle positions. Solving this exactly via linear programming is computationally expensive and, more importantly, non-differentiable with respect to the inputs.

The key insight from [11] is that adding a discrete entropy regularizer to the objective,

$$\min_{\mathbf{P} \in \mathcal{U}(\mathbf{a}, \mathbf{b})} \langle \mathbf{C}, \mathbf{P} \rangle - \epsilon \mathbf{H}(\mathbf{P}), \quad \mathbf{H}(\mathbf{P}) = - \sum_{i,j} P_{ij} (\log P_{ij} - 1), \quad (95)$$

makes the problem ϵ -strongly convex, yielding a unique solution that is differentiable with respect to $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{C}$. By the KKT conditions, the optimal $\mathbf{P}^*$ must have the Gibbs-kernel form:

$$P_{ij}^* = u_i K_{ij} v_j, \quad K_{ij} = \exp\left(-\frac{C_{ij}}{\epsilon}\right), \quad (96)$$

where $\mathbf{u} \in \mathbb{R}^N$ and $\mathbf{v} \in \mathbb{R}^N$ are two unknown scaling vectors. To determine them, we substitute this form back into the two marginal constraints $\mathbf{P}^* \mathbf{1}_N = \mathbf{a}$ and $(\mathbf{P}^*)^\top \mathbf{1}_N = \mathbf{b}$, obtaining the equivalent fixed-point conditions:

$$\mathbf{u} \odot (\mathbf{K} \mathbf{v}) = \mathbf{a}, \quad \mathbf{v} \odot (\mathbf{K}^\top \mathbf{u}) = \mathbf{b}, \quad (97)$$

where $\odot$ and $\oslash$ denote element-wise multiplication and division. Rearranging, we can solve for $\mathbf{u}$ and $\mathbf{v}$ alternately:

$$\mathbf{u}^{(\ell)} = \mathbf{a} \oslash (\mathbf{K} \mathbf{v}^{(\ell-1)}), \quad \mathbf{v}^{(\ell)} = \mathbf{b} \oslash (\mathbf{K}^\top \mathbf{u}^{(\ell)}), \quad (98)$$

initialized at $\mathbf{v}^{(0)} = \mathbf{1}_N$. Upon convergence, the optimal coupling is recovered as $\mathbf{P}^{(L)} = \text{diag}(\mathbf{u}^{(L)}) \cdot \mathbf{K} \cdot \text{diag}(\mathbf{v}^{(L)})$. This is the vanilla Sinkhorn algorithm.

The practical difficulty is numerical: since $K_{ij} = \exp(-C_{ij}/\epsilon)$, when particle positions are spread out or ϵ is small, the entries of $\mathbf{K}$ become extremely small, and the repeated divisions in (98) cause the

scaling variables $\mathbf{u}$ and $\mathbf{v}$ to grow in the exponential domain, producing Inf/NaN overflow. This is the well-known numeric instability of the vanilla Sinkhorn. The standard remedy is to absorb the scaling into dual potentials $\alpha = \epsilon \log \mathbf{u}$ and $\beta = \epsilon \log \mathbf{v}$, so that the coupling takes the form

$$T_{ij} = \exp\left(\frac{\alpha_i + \beta_j - C_{ij}}{\epsilon}\right), \quad (99)$$

and the marginal-matching updates (98) turn into log-sum-exp operations. These can be stabilized numerically by subtracting the row or column minimum before exponentiating — this is the log-stabilized Sinkhorn, whose iteration is given in the algorithm below.

Algorithm 15 Sinkhorn Iteration (Dual Potentials)

Require: Cost matrix $\mathbf{C} \in \mathbb{R}^{N \times N}$, $C_{ij} = \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2/2$

Require: Log-weights $\log w^{(i)}$, regularization $\epsilon > 0$, max iterations, threshold δ

- 1: Initialize dual potentials: $\alpha^{(i)} \leftarrow 0$, $\beta^{(i)} \leftarrow 0$ for $i = 1, \dots, N$
 - 2: $\log u \leftarrow -\log N$ (uniform target: $1/N$)
 - 3: **repeat**
 - 4: $\alpha_{\text{new}}^{(i)} \leftarrow -\epsilon \log \sum_j \exp\left(\frac{\log w^{(j)} + \beta^{(j)} - C_{ji}}{\epsilon}\right)$ (softmin over columns)
 - 5: $\beta_{\text{new}}^{(j)} \leftarrow -\epsilon \log \sum_i \exp\left(\frac{\log u + \alpha_{\text{new}}^{(i)} - C_{ij}}{\epsilon}\right)$ (softmin over rows)
 - 6: $\alpha \leftarrow \frac{1}{2}(\alpha + \alpha_{\text{new}})$, $\beta \leftarrow \frac{1}{2}(\beta + \beta_{\text{new}})$ (damping for stability)
 - 7: **until** $\max(|\Delta\alpha|, |\Delta\beta|) < \delta$ or max iterations reached
 - 8: **Extrapolation** (for differentiability via implicit function theorem):
 - 9: $\alpha_{\text{sg}} \leftarrow \text{stop_gradient}(\alpha)$, $\beta_{\text{sg}} \leftarrow \text{stop_gradient}(\beta)$
 - 10: One more Gauss-Seidel step: $\alpha \leftarrow \text{softmin}(\epsilon, \mathbf{C}^\top, \log \mathbf{w} + \beta_{\text{sg}}/\epsilon)$
 - 11: $\beta \leftarrow \text{softmin}(\epsilon, \mathbf{C}, \log \mathbf{u} + \alpha/\epsilon)$
 - 12: **return** α, β
-

Lines 12–15 of the algorithm above are the *extrapolation trick* for approximate differentiability. The converged potentials (α, β) are detached from the computation graph via `stop_gradient`, and one additional Gauss-Seidel step is taken on the live graph. This gives autodiff a single-step window into how the potentials depend on the inputs $(\mathbf{C}, \log \mathbf{w})$, without differentiating through all the preceding iterations. We will see in Section 5.5 that this is only an approximation—accurate for weight gradients but biased for particle-position gradients—and how implicit differentiation provides the correct alternative.

Algorithm 16 Entropy-Regularized Optimal Transport Resampling

Require: Particles $\{\mathbf{x}^{(i)}, w^{(i)}\}_{i=1}^N$, regularization ϵ

- 1: **Normalize particles:** $\tilde{\mathbf{x}}^{(i)} \leftarrow (\mathbf{x}^{(i)} - \bar{\mathbf{x}})/s$ where $\bar{\mathbf{x}}$ = mean, $s = \text{std} \cdot \sqrt{d}$
 - 2: Compute cost matrix: $C_{ij} = \|\tilde{\mathbf{x}}^{(i)} - \tilde{\mathbf{x}}^{(j)}\|^2/2$
 - 3: Solve dual potentials: $(\alpha, \beta) \leftarrow \text{Sinkhorn}(\log \mathbf{w}, \mathbf{C}, \epsilon)$
 - 4: **Transport matrix:**
 - 5: $\log T_{ij} \leftarrow (\alpha_i + \beta_j - C_{ij})/\epsilon$
 - 6: Normalize columns: $T_{ij} \leftarrow T_{ij} / \sum_k T_{kj}$ (each column sums to $1/N$)
 - 7: Apply source weights: $T_{ij} \leftarrow T_{ij} \cdot w^{(j)}$ (rows marginalize to $\mathbf{w}$)
 - 8: **Transport particles:**
 - 9: $\hat{\mathbf{x}}^{(i)} \leftarrow \sum_j T_{ij} \mathbf{x}^{(j)}$ (matrix-vector product)
 - 10: $\hat{w}^{(i)} \leftarrow 1/N$ for all i (uniform weights after transport)
 - 11: **return** $\{\hat{\mathbf{x}}^{(i)}, \hat{w}^{(i)}\}_{i=1}^N$, transport matrix $\mathbf{T}$
-

5.4 The Derivative Chain through Resampling

The core difficulty in differentiating through a particle filter is the resampling step. Everything else—the transition model $p(x_k | x_{k-1}, \theta)$, the observation likelihood $p(z_k | x_k, \theta)$, and the weight computation—is differentiable. Resampling is the step where discrete particle selection breaks the gradient flow. To understand what each method does and does not give us, we need to trace the derivative chain explicitly.

At time step k , the negative log-likelihood contribution is $-\log \sum_i \tilde{w}_k^{(i)}$ where the unnormalized weights $\tilde{w}_k^{(i)} = w_{k-1}^{(i)} \cdot p(z_k | x_k^{(i)}, \theta)$ depend on θ both through the observation model and through the

particle positions $x_k^{(i)}$. Particle $x_k^{(i)}$ was sampled from $p(x_k | x_{k-1}^{(j_i)}, \theta)$, where j_i is the index selected by resampling at step $k - 1$. The derivative $\partial x_k^{(i)} / \partial \theta$ therefore has two contributions: the direct effect through the transition model (differentiable) and the indirect effect through which ancestor j_i was selected (depends on how resampling is done).

Systematic resampling with stop gradient. We treat the selected particle indices as constants and detach the gradient at the resampling step. The derivative flows only through the weights after resampling and the transition model, but sees nothing about how θ affects particle selection. The gradient is:

$$\frac{\partial L}{\partial \theta} = - \sum_k \sum_i \frac{\partial \log p(z_k | x_k^{(i)}, \theta)}{\partial \theta} + \frac{\partial \log p(x_k^{(i)} | x_{k-1}^{(j_i)}, \theta)}{\partial \theta} \quad (100)$$

where $x_{k-1}^{(j_i)}$ is treated as a fixed constant. This *selection gradient* is zero. The resulting gradient is biased—it misses the signal from particle selection—but bounded and continuous within each particle configuration. This makes it safe for both SGD and (in simple models) HMC.

Soft resampling. Soft resampling [22] blends the weights with a uniform distribution before selection: $\tilde{w}_i = \alpha w_i + (1 - \alpha)/N$. Particles are still selected discretely from $\tilde{w}_i$, but an importance correction weight $w_i / \tilde{w}_i$ is applied after selection and is differentiable with respect to θ through w_i . The gradient now flows through this correction factor in addition to the direct observation and transition terms. It does not flow through which particle was selected. Soft resampling is therefore partially differentiable—richer gradient information than stop gradient, but the selection is still invisible.

OT resampling. Optimal transport resampling [10] replaces discrete index selection entirely with a continuous transport map. Now that we have defined the transport matrix T from the Sinkhorn algorithm, the resampled particles are the weighted average:

$$\tilde{x}_k^{(i)} = \sum_j T_{ij} x_k^{(j)}. \quad (101)$$

There is no discrete selection—every input particle contributes to every output particle through T_{ij} . The key point is that T is itself a function of both the particle positions $\mathbf{x}_k$ (through the cost matrix $C_{ij} = \|\tilde{x}^{(i)} - \tilde{x}^{(j)}\|^2/2$) and the weights $\mathbf{w}_k$ (through the Sinkhorn marginal constraints). The total derivative of the resampled particles with respect to θ therefore has three terms, which we write schematically as:

$$\frac{d\tilde{\mathbf{x}}_k}{d\theta} = \underbrace{\frac{\partial T}{\partial \mathbf{x}_k} \frac{d\mathbf{x}_k}{d\theta} \cdot \mathbf{x}_k}_{\text{Term 1: particle geometry}} + \underbrace{T \cdot \frac{d\mathbf{x}_k}{d\theta}}_{\text{Term 2: direct transport}} + \underbrace{\frac{\partial T}{\partial \mathbf{w}_k} \frac{d\mathbf{w}_k}{d\theta} \cdot \mathbf{x}_k}_{\text{Term 3: weight gradient}} \quad (102)$$

This is a schematic chain rule—the object $\partial T / \partial \mathbf{x}_k$ is a higher-order tensor and the contractions are suppressed for clarity. The three paths are:

- **Term 1 (particle geometry):** A change in θ changes particle positions, which changes the cost matrix C , which changes the Sinkhorn potentials, which changes T , which changes the transported particles.
- **Term 2 (direct transport):** A change in θ changes particle positions, which are directly transported by the (fixed) transport matrix T .
- **Term 3 (weight gradient):** A change in θ changes the observation likelihood, which changes the weights, which changes T , which changes the transported particles. This is the *selection gradient* that stop-gradient and soft resampling miss entirely.

Together, these three paths give a gradient that is correct in principle. The regularized OT map is smooth, so the loss surface is continuous. The practical issue is computing this gradient faithfully through the iterative Sinkhorn solver, which we address in Section 5.5.

A summary of the gradient properties of each method is given in Table 1.

Table 1: Resampling strategies for differentiable particle filters.

Resampling	Fully diff.?	Selection gradient	Gradient bounded?	Loss continuous?
Systematic (stop grad)	No	None	Yes	No
Soft	Partial	Partial	Yes	No
OT Sinkhorn	In principle	Full	Not guaranteed	Yes

Implications for HMC. HMC’s leapfrog integrator treats the gradient as a deterministic force field. Two properties are critical: the gradient must be bounded (a single spike can send the trajectory to infinity and waste the proposal) and the loss surface must be continuous (discontinuities in $U(\theta)$ cause large ΔH and rejected proposals). Systematic resampling satisfies the first but not the second, because small changes in θ can flip a particle across a resampling threshold, causing a jump in the log-likelihood. OT resampling satisfies both: the transport map is smooth, so the loss surface is continuous, and there is no threshold discontinuity. There is a subtlety: even with OT resampling, conditional resampling (triggered when ESS drops below a threshold) introduces a second source of non-smoothness—the branch between “resample” and “don’t resample” is itself a discontinuity. OT makes this much less severe because the ESS stays closer to N when transport is smooth, but it does not eliminate it entirely. For simple models where the loss surface is already mild (e.g., linear Gaussian), systematic resampling can work fine with HMC. For nonlinear models with strong particle diversity effects, OT resampling is the right choice.

5.5 Approximate vs Implicit Sinkhorn Backward

The entropy-regularized OT map is differentiable in principle. The practical question is: how do we compute the backward pass through the Sinkhorn iteration? There are two approaches, and understanding their difference is essential for getting correct gradients in the differentiable particle filter pipeline.

5.5.1 The Extrapolation Approximation

The approach used in our initial implementation (and described in the Sinkhorn algorithm above) is to run the iteration to convergence, detach the converged potentials from the computation graph, and take one more Gauss–Seidel step on the live graph:

$$\alpha_{\text{sg}} = \text{stop_gradient}(\alpha_{\text{converged}}), \quad \beta_{\text{sg}} = \text{stop_gradient}(\beta_{\text{converged}}), \quad (103)$$

$$\alpha_{\text{new}} = \text{softmin}(\epsilon, \mathbf{C}^\top, \log \mathbf{w} + \beta_{\text{sg}}/\epsilon), \quad (104)$$

$$\beta_{\text{new}} = \text{softmin}(\epsilon, \mathbf{C}, \log \mathbf{u} + \alpha_{\text{new}}/\epsilon). \quad (105)$$

Autodiff sees only this final step. The true derivative of the converged potentials with respect to the inputs $(\mathbf{C}, \log \mathbf{w})$ would require differentiating through all iterations. The one-step window is an approximation.

This approximation is accurate for **Term 3** (the weight gradient), because the weights enter linearly into the marginal constraint and the one-step update captures their effect well. But it is biased for **Term 1** (the particle geometry gradient), because particle positions affect the cost matrix $\mathbf{C}$, which feeds into every Sinkhorn iteration. The single-step window cannot capture how the potentials depend on $\mathbf{C}$ across the full iteration history.

When does this matter? It depends on whether $d\mathbf{x}_k/d\theta$ is zero or not. For a Bootstrap Particle Filter (BPF) on a linear Gaussian model with $\theta = \sigma_{\text{obs}}$, the particles come from the state transition $x_k^{(i)} = F\tilde{x}_{k-1}^{(i)} + B\epsilon_k^{(i)}$, which does not depend on θ . So $d\mathbf{x}_k/d\theta = 0$, Terms 1 and 2 vanish, and only Term 3 survives. The extrapolation approximation works well: we observe an autodiff-to-numerical-derivative ratio of approximately 1.08. For LEDH (Local Exact Daum–Huang), the flow matrices depend on θ through $R = \sigma_{\text{obs}}^2 DD^\top$, so $d\mathbf{x}_k/d\theta \neq 0$ and all three terms are active. The extrapolation approximation produces a ratio of approximately 1.33—a clear bias from Term 1. These ratios are empirical validation targets on a specific model, not universal constants, but they sharply illustrate the problem.

5.5.2 Implicit Differentiation at the Fixed Point

The proper fix is implicit differentiation. At convergence, the Sinkhorn potentials satisfy a system of fixed-point equations (the marginal constraints). By the implicit function theorem, we can compute the

exact Jacobian of the converged potentials with respect to the inputs without differentiating through the iteration [?].

The idea is as follows. Let $\mathbf{f}$ and $\mathbf{g}$ denote the converged dual potentials (absorbing the log-scaling), and let $P_{ij} = \exp((f_i + g_j - C_{ij})/\epsilon)$ be the transport plan at the fixed point. Given an upstream gradient $\partial L/\partial T$, we need the vector-Jacobian product (VJP) $\partial L/\partial \mathbf{C}$ and $\partial L/\partial \log \mathbf{w}$. The implicit VJP reduces to solving a $(2N - 1) \times (2N - 1)$ linear system:

$$\mathbf{B}\boldsymbol{\mu} = \mathbf{r}, \quad (106)$$

where $\mathbf{B}$ is formed from the transport plan P and the source/target marginals $\mathbf{a}, \mathbf{b}$:

$$\mathbf{B} = \begin{pmatrix} \text{diag}(\mathbf{a}_{1:N-1}) & P_{1:N-1,:} \\ P_{1:N-1,:}^\top & \text{diag}(\mathbf{b}) \end{pmatrix} + \rho \mathbf{I}, \quad (107)$$

and $\mathbf{r}$ is the right-hand side constructed from the upstream gradient $\partial L/\partial T$. The system is rank-deficient by 1 because the potentials are defined up to a global constant (a gauge freedom), so we drop one equation and add a small ridge ρ for numerical stability. Once $\boldsymbol{\mu}$ is solved, the gradient with respect to the cost matrix is

$$\frac{\partial L}{\partial C_{ij}} = -\frac{P_{ij}}{\epsilon} \left(\frac{\partial L}{\partial T_{ij}} - \mu_i^{(f)} - \mu_j^{(g)} \right), \quad (108)$$

and the gradient with respect to the log-weights follows from the marginal constraint.

The computational cost of the dense solve is $\mathcal{O}(N^3)$, which is acceptable for $N = 200$ particles (roughly $10\times$ the cost of the forward Sinkhorn backward). For $N = 1000$ and above, a conjugate gradient (CG) solver with Jacobi preconditioning brings the cost down to $\mathcal{O}(kN^2)$ with $k \approx 50$ iterations, which is comparable to one extra Sinkhorn forward pass. The expected outcome is that the autodiff-to-numerical ratio moves from ~ 1.33 toward ~ 1.0 for LEDH, confirming that the particle-geometry gradient (Term 1) is now computed faithfully.

5.6 Neural Acceleration

With the Sinkhorn backward producing faithful gradients, the OT resampling pipeline is fully differentiable. The remaining bottleneck is computational: Sinkhorn costs $\mathcal{O}(LN^2)$ per resampling step, where L is the number of iterations.

More recently, people also used neural networks to solve the Monge problem itself [12]. GradNetOT exploits Brenier’s theorem, which says that under certain conditions (continuous densities, squared Euclidean cost), the Monge and Kantorovich problems are equivalent, and the unique transport map $T(x)$ is the gradient of a convex function $\phi(x)$, where $\phi(x)$ satisfies the Monge-Ampère equation

$$p(x) = q(\nabla \phi(x)) \det(H_\phi(x)) \quad (109)$$

where $H_\phi = J_T$ is the $d \times d$ positive semi-definite Hessian of ϕ . But this architecture is not directly applicable to resampling because the transformation will push the continuous distribution $p(x)$ to the target distribution $q(y)$. We will discuss the possible way to make it work.

We specialize the Brenier setup to the resampling problem. Let $\{x_i\}_{i=1}^N$ be the current particles and $\{w_i\}_{i=1}^N$ their normalized weights. Smooth both empirical measures with the same Gaussian kernel K_h of scalar bandwidth h ,

$$p_h(x) = \frac{1}{N} \sum_{i=1}^N K_h(x - x_i), \quad q_h(x) = \sum_{i=1}^N w_i K_h(x - x_i), \quad (110)$$

so p_h is the source — the equally weighted particle cloud — and q_h is the target — the same particles weighted by the w_i . The transport map $T : \mathbb{R}^d \rightarrow \mathbb{R}^d$ we want to learn satisfies the Monge-Ampère equation in this notation,

$$p_h(x) = q_h(T(x)) \det J(x), \quad J(x) = \nabla T(x). \quad (111)$$

Once T is known, resampling is one forward pass: $x'_i = T(x_i)$ with uniform weights $1/N$. The original particles are themselves samples from p_h , so applying T pushes them into samples of q_h .

The standard route to Brenier maps is to parameterize the convex potential ϕ with an input-convex neural network and use $T = \nabla \phi$. Training requires two derivatives of ϕ : first with respect to x to obtain

T , then with respect to x again to obtain $J = \nabla^2 \phi$, because the loss depends on $\det J$. Backpropagation therefore passes through a second derivative of the potential, and the convexity of ϕ is only approximate in practice, so an additional penalty is needed to keep $\nabla^2 \phi$ positive semi-definite. We avoid both costs by parameterizing T directly with a modular architecture, made conditional on the cloud through a context vector c produced by a permutation-invariant set encoder. Each module has a pre-activation u_m and an output T_m ,

$$u_m(x; c) = W_m x + U_m c + b_m(c), \quad (112)$$

$$T_m(x; c) = W_m^\top \phi(\text{softplus}(\beta_m(c)) u_m(x; c)), \quad (113)$$

where W_m is a state-independent learned matrix, U_m is a context-mixing matrix, $b_m(c)$ and $\beta_m(c)$ are scalar shifts produced from c by linear layers, and ϕ is a smooth scalar activation applied element-wise. The full map adds these modules to the identity with positive gates,

$$T(x; c) = x + d(c) + \sum_m \text{softplus}(\alpha_m(c)) T_m(x; c). \quad (114)$$

Holding c fixed, the module Jacobian and the full Jacobian with respect to x are

$$J_m(x; c) = \text{softplus}(\beta_m(c)) W_m^\top \text{diag}(\phi'(\text{softplus}(\beta_m(c)) u_m(x; c))) W_m, \quad (115)$$

$$J(x; c) = I + \sum_m \text{softplus}(\alpha_m(c)) J_m(x; c). \quad (116)$$

Each summand has the form $W^\top D W$ with diagonal $D \succeq 0$, since ϕ' is non-negative for monotone smooth ϕ . The full Jacobian is therefore symmetric positive semi-definite by construction. There is no Hessian, no nested differentiation, and no convexity penalty. The matrices W_m are held state-independent on purpose: only the scalar gates $\alpha_m(c), \beta_m(c)$ and the shifts $b_m(c), d(c)$ depend on c , which preserves the $W^\top D W$ structure for every cloud.

The choice of activation ϕ is load-bearing. The training loss involves $\log \det J(x; c)$ and the downstream local covariance transport in our LEDH variant uses $J \Sigma J^\top$, so $J(x; c)$ — the Jacobian with respect to x for fixed c — must exist everywhere and vary smoothly in x . ReLU is only C^0 : its derivative is a step function, and a ReLU-based map would give a Jacobian with discontinuities along every hyperplane $W_m x + b_m = 0$. We use the weighted hyperbolic tangent $\phi(z) = \gamma \tanh(z)$, which is C^∞ , so $J(x; c)$ varies smoothly in x as required. The set encoder uses ReLU activations internally, but its output c is held fixed at inference, so encoder ReLUs do not affect the smoothness of J in x .

The training objective is the squared pointwise residual of the Monge-Ampère equation. From the change-of-variables identity $\log p_h(x) = \log q_h(T(x)) + \log \det J(x)$ we define the residual and its mean square,

$$r(x; c) = \log \det J(x; c) - (\log p_h(x) - \log q_h(T(x; c))), \quad L_{\text{MA}} = \mathbb{E}_{x \sim p_h} [r(x; c)^2]. \quad (117)$$

The expectation is estimated by sampling collocation points from p_h . The loss is self-supervised: there is no external transport teacher, in particular no Sinkhorn-supervised target. Sinkhorn solves the discrete entropic optimal transport problem, which is a different and biased approximation; imitating it would import that bias rather than the continuous Brenier map we want. To keep J well-conditioned and smooth on the kernel scale we combine four mechanisms: residual initialization, with α_m initialized so that $\text{softplus}(\alpha_m) \approx 0$ at step zero so that $T \approx \text{id}$ and $J \approx I$ before training begins; a local-linearity penalty $\mathbb{E}[\|T(x + \delta) - T(x) - J(x) \delta\|^2]$ with $\delta \sim K_h$, which directly penalizes J varying fast on the kernel scale; a soft hinge on the smallest eigenvalue $\lambda_{\min}(J(x))$ that prevents J from collapsing; and bandwidth annealing, in which h is decreased over training so the operator solves a sequence of progressively sharper transport problems. The training distribution matters as much as the loss. Synthetic particle clouds are useful only if they reproduce the relation between position and weight that the operator will see at inference. In a particle filter, the weights are induced by the likelihood and are therefore correlated with position, so training mainly on clouds with position-independent weights is not sufficient. The cleanest training data comes directly from runs of the target models. Questions of cross-model generalization, high-dimensional scaling, and end-to-end gradient quality through the compiled filter are deferred to the implementation and benchmarking sections.

In addition, there are of course alternatives to OT resampling. Notable example is Particle Transformer [13, 14], where we replace the OT resampler with a Transformer mechanism (self-attention) to interact particles with each other and re-weight them differentiably. To make this review more focused,

we will not discuss it further. The key idea is that you demand the transformer to output particles with equal weights. Therefore, it learns from the data to put particles into the right positions.

Optimal transport resampling out-performs in certain scenarios but its real power lies in that it makes it possible for the entire filtering architecture to be differentiable. Because one can easily use neural networks to model the dynamics or observation, which has been extensively discussed in [15]. I think there is no point to make a comprehensive review here as it is problem specific and requires careful testing, there is no one-fits-all solution. This field is exploding and there are so many different architectures that one can try. As pointed out in the Bonus question 2, it will be nice that we don't have to solve the optimal transport problem repeatedly, because each resampling is a different set of initial particles and weights. One way to deal with this problem is amortized optimization [16], where you train a model to predict solutions to optimization problems rather than solving each problem from scratch. Its application to the OT problem is known as Meta-OT [17]. The idea is actually very easy: when we run a filtering algorithm, we need to solve the resampling a lot of times. Each time, we will have a transport schedule from the dual problem. We simply run the filter a lot of times and record what kind of transport plan we will run into in such problems. Then we train a neural network to make predictions.

For the purpose of building a differentiable particle filter, we start with entropy-regularized OT resampling. It is one more step on top of our regular filter while fully differentiable. This is a good place for benchmarking and ensuring the entire pipeline works. Any further improvement or innovative algorithm, namely neural network based algorithms, are fully differentiable from the beginning. But they require additional training, for example Meta-OT. As for neural operator based approach, it will also require additional training for solving the Monge-Ampère equation. The idea is to work in parallel: first build the differentiable particle filter pipeline with model-agnostic entropy-regularized OT resampling, then for specific models, we can improve on the existing architecture with all the logistic components ready.

6 Parameter Estimation with Differentiable Particle Filters

6.1 Markov Chain Monte Carlo

6.1.1 Metropolis-Hastings

Our goal is to estimate the model parameters θ from observations $z_{1:T}$. We want either the MAP point or the full posterior $p(\theta \mid z_{1:T})$. The particle filter gives us the marginal likelihood $p(z_{1:T} \mid \theta)$, and combined with a prior $p(\theta)$, we have access to the log-posterior up to a constant:

$$\log p(\theta \mid z_{1:T}) = \log p(z_{1:T} \mid \theta) + \log p(\theta) + \text{const.} \quad (118)$$

The question is how to explore this surface, the geometric surface spanned by θ , efficiently.

The simplest approach is a random walk in parameter space. Let $L(\theta) = -\log p(\theta \mid z_{1:T})$ denote the negative log-posterior. Propose a new parameter $\theta' = \theta + w$ where w is a random vector, compute the change $\Delta L = L(\theta') - L(\theta)$, and always accept moves that improve the posterior ($\Delta L < 0$). The question is what to do when $\Delta L > 0$. Simply rejecting all uphill moves would trap the chain at the nearest local minimum. The Metropolis-Hastings algorithm [18] accepts the proposed move with probability

$$\alpha = \min(1, e^{-\Delta L}) = \min\left(1, \frac{p(\theta' \mid z_{1:T})}{p(\theta \mid z_{1:T})}\right). \quad (119)$$

Even uphill moves get accepted with probability $e^{-\Delta L}$, allowing the chain to escape shallow local minima. This acceptance rule satisfies detailed balance, which guarantees the chain converges to the correct target distribution $p(\theta \mid z_{1:T})$.

The fundamental limitation with random-walk MH runs at two levels.

1. The first is just finding the right region. Even with three parameters, the posterior does not fill the whole parameter space — it concentrates on a low-dimensional surface, a thin shell inside the 3D cube. This is the *typical set*. Most of the volume is empty; the posterior mass lives on a much smaller manifold. In higher dimensions this only gets worse: the total volume grows exponentially while the typical set stays low-dimensional, so the fraction of space carrying any meaningful probability shrinks exponentially. A random walk has no way to seek out that manifold — it just stumbles around hoping to land on it.
2. The second problem shows up even after the chain has found the typical set. The step size ε in the proposal $\theta' = \theta + \varepsilon \xi$, $\xi \sim \mathcal{N}(0, I)$, is the radius of the proposal ball. To see why ε must shrink with dimension, consider a simple illustrative case: d independent standard Gaussian coordinates, $\log p(\theta) = -\frac{1}{2}\|\theta\|^2 + \text{const.}$ The change in log-posterior for coordinate i after a step $\varepsilon \xi_i$ is exactly

$$\Delta \log p_i = -\varepsilon \theta_i \xi_i - \frac{\varepsilon^2 \xi_i^2}{2}. \quad (120)$$

The first term has mean zero and standard deviation $\varepsilon |\theta_i| \sim \varepsilon$ for a typical $\theta_i \sim \mathcal{N}(0, 1)$. The second term has mean $-\varepsilon^2/2$ since $\mathbb{E}[\xi_i^2] = 1$. Summing over all d coordinates, the total change in log-posterior has

$$\mathbb{E}[\Delta \log p] = -\frac{\varepsilon^2 d}{2}, \quad \text{std}[\Delta \log p] \sim \varepsilon \sqrt{d}. \quad (121)$$

For the acceptance rate not to collapse, the mean must not be overwhelmingly negative: $\varepsilon^2 d = \mathcal{O}(1)$, which gives $\varepsilon = \mathcal{O}(d^{-1/2})$. The step size shrinks not because any single direction is narrow, but because perturbations accumulate across all d dimensions at once. With $\varepsilon \sim d^{-1/2}$, the chain takes $\mathcal{O}(d)$ steps to move a meaningful distance — exploration becomes quadratically more expensive as dimension grows.

6.1.2 Hamiltonian Monte Carlo

When the pipeline is fully differentiable, we can do much better. Hamiltonian Monte Carlo (HMC) [19, 20] treats parameter estimation as a physical dynamics problem. We introduce an auxiliary momentum variable $\mathbf{p}$ of the same dimension as θ and define the Hamiltonian (total energy):

$$H(\mathbf{p}, \theta) = \underbrace{\frac{1}{2} \mathbf{p}^\top M^{-1} \mathbf{p}}_{K(\mathbf{p}) \text{ (kinetic)}} + \underbrace{U(\theta)}_{\text{(potential)}} \quad (122)$$

where $U(\theta) = -\log p(\theta \mid z_{1:T})$ is the negative log-posterior and M is the mass matrix. Think of a ball rolling in a potential landscape: the ball picks up speed going downhill and slows going uphill, efficiently exploring the landscape while conserving total energy. Hamilton’s equations describe the trajectory:

$$\frac{d\theta}{dt} = M^{-1}\mathbf{p}, \quad \frac{d\mathbf{p}}{dt} = -\nabla_{\theta}U(\theta). \quad (123)$$

These trajectories conserve H exactly, so the ball naturally curves around the posterior contours rather than wandering randomly.

Leapfrog integration. In practice we integrate Hamilton’s equations numerically with the *leapfrog* (Störmer-Verlet) integrator, which alternates half-step momentum updates and full-step position updates. Concretely, with step size ε and L steps:

$$\begin{aligned} \mathbf{p}_{t+\frac{1}{2}} &\leftarrow \mathbf{p}_t - \frac{\varepsilon}{2}\nabla_{\theta}U(\theta_t) \\ \theta_{t+1} &\leftarrow \theta_t + \varepsilon M^{-1}\mathbf{p}_{t+\frac{1}{2}} \\ \mathbf{p}_{t+1} &\leftarrow \mathbf{p}_{t+\frac{1}{2}} - \frac{\varepsilon}{2}\nabla_{\theta}U(\theta_{t+1}) \end{aligned}$$

The leapfrog scheme is symplectic, meaning it approximately conserves H even over many steps. Each HMC iteration samples a fresh momentum $\mathbf{p} \sim \mathcal{N}(0, M)$, runs L leapfrog steps to produce a proposal $(\theta^*, \mathbf{p}^*)$, then accepts with the Metropolis criterion:

$$\alpha = \min\left(1, e^{-(H(\mathbf{p}^*, \theta^*) - H(\mathbf{p}, \theta))}\right). \quad (124)$$

Because the leapfrog nearly conserves H , the energy difference $\Delta H \approx 0$ and the acceptance probability is close to 1, even for proposals far from the current point. This is the key advantage over random-walk MH: the momentum carries the state along energy contours, allowing large, directed moves.

Step size and tuning. The leapfrog integrator approximates the continuous Hamiltonian trajectory with finite steps of size ε . If ε is too large, the numerical integration accumulates error, H is no longer conserved, and ΔH becomes large — the Metropolis correction then rejects most proposals and the chain stalls. If ε is too small, each accepted step moves only a tiny distance and the chain explores slowly. The acceptance rate is therefore a direct readout of whether ε is well-calibrated: too low means overshooting; too high means nearly standing still.

In our implementation, we initialize $\varepsilon = 0.01$ and let it self-adapt during the first 80% of burn-in iterations via dual averaging [21], which adjusts ε to hit a target acceptance rate of 65%. After the adaptation window closes, ε is fixed for the remainder of the run. We also monitor the gradient norm $\|\nabla U\|$ at each leapfrog step: a gradient that spikes or produces NaN is not a tuning problem — it signals numerical instability somewhere in the model or filter pipeline.

Mass matrix. To build intuition, start from a well-chosen coordinate system. In physics, for a system that is not over-constrained, the right parameterization decouples the dynamics into independent trajectories. Think of describing a circle: polar coordinates (r, ϕ) decouple the radial and angular motion immediately, while Cartesian (x, y) keep them entangled. The same principle applies here — if the posterior directions are strongly coupled or have very different scales, the ball’s trajectory mixes up slow and fast modes and explores inefficiently.

The mass matrix M is how we implement this rescaling within HMC without changing the parameterization itself. At each iteration we sample a fresh momentum $\mathbf{p} \sim \mathcal{N}(0, M)$, so M_{ii} controls how much kinetic energy is injected in direction i before the ball starts rolling. Think of each parameter direction as an independent harmonic oscillator — some with a steep potential (narrow posterior, high curvature) and some with a flat potential (wide posterior, low curvature). A steep direction needs more initial kinetic energy to escape the narrow well and explore; a flat one needs less. Setting M_{ii} proportional to the posterior variance in direction i balances the energy across modes so that all directions are explored at a similar rate.

In our implementation, we use a fixed diagonal mass matrix specified as a vector of per-parameter values. The default is the identity ($M = I$), and the values are set manually based on the expected scale of each parameter. A full production setup would run a short pilot chain, estimate the empirical per-parameter variance, and set the diagonal entries from that — but for the benchmarks here, the parameters are well-understood enough that the scales can be set directly.

6.2 MAP as a Pipeline Smoke Test

Before running HMC, it is always a good idea to run MAP estimation first. MAP finds the mode of the posterior by gradient descent on $U(\theta) = -\log p(\theta \mid z_{1:T})$,

$$\theta \leftarrow \theta - \eta \nabla_{\theta} U(\theta), \quad (125)$$

using Adam or SGD with a fresh particle filter seed at each step, making the gradient stochastic. MAP is far more forgiving than HMC in two ways. First, it does not need the loss surface to be smooth: stochastic gradients average out over many steps, so occasional discontinuities from resampling events do not derail convergence. Second, it does not need the gradient to be unbiased or to carry full information—as long as the gradient points roughly downhill on average, the chain converges. This means MAP works with any resampling method, including the simplest stop-gradient systematic resampling.

The reason this matters is diagnostic: if MAP fails—loss diverges, gradients blow up, or the parameters drift away from the truth—there is a numerical problem somewhere in the model or the filter pipeline. MAP is therefore our first benchmark before we ever run HMC. We treat MAP convergence on a known model (e.g., 1-D linear Gaussian) as the minimum bar that any new filter configuration must clear. Only once MAP works cleanly do we move to HMC, which has the additional requirements of a deterministic loss surface (fixed particle seed) and smooth gradients.

6.3 Differentiable Resampling and the Gradient Chain

The core difficulty in differentiating through a particle filter is the resampling step. We have analyzed the derivative chain through resampling in detail in Section 5.4, showing that entropy-regularized OT resampling provides a fully differentiable (in principle) alternative to systematic and soft resampling. Here we focus on the consequences for parameter estimation.

6.4 Benchmark: Linear Gaussian Model

We use the linear Gaussian model as the primary validation target. The Kalman filter gives the exact posterior, so we have ground truth for both the filter estimates and the parameter posterior.

6.4.1 MCMC Diagnostics

After every HMC run we report four diagnostics.

Acceptance rate. At each HMC step, a proposal θ^* is drawn by integrating Hamilton’s equations for L leapfrog steps from the current state. The Metropolis acceptance probability is

$$\alpha = \min(1, e^{-\Delta H}), \quad \Delta H = H(\mathbf{p}^*, \theta^*) - H(\mathbf{p}, \theta). \quad (126)$$

If the leapfrog integrator were exact, $\Delta H = 0$ and every proposal would be accepted. Numerical discretisation error causes $\Delta H \neq 0$, so some proposals are rejected. The acceptance rate is therefore a direct diagnostic for step size calibration: too low means the step size is too large and the trajectory overshoots; too high (near 100%) means the step size is too small and the chain barely moves. We target 65% acceptance, which is the theoretically optimal rate for high-dimensional Gaussian targets [23].

Step size and dual averaging. The leapfrog step size ε is adapted during the first 80% of burn-in using dual averaging [21]. Let δ be the target acceptance rate and α_m the actual acceptance rate at step m . The update equations are:

$$\bar{H}_m = \left(1 - \frac{1}{m + t_0}\right) \bar{H}_{m-1} + \frac{1}{m + t_0} (\delta - \alpha_m), \quad (127)$$

$$\log \varepsilon_m = \mu - \frac{\sqrt{m}}{\gamma} \bar{H}_m, \quad (128)$$

$$\log \bar{\varepsilon}_m = m^{-\kappa} \log \varepsilon_m + (1 - m^{-\kappa}) \log \bar{\varepsilon}_{m-1}. \quad (129)$$

Here $\mu = \log(10\varepsilon_0)$ is the reference target (set to $10\times$ the initial step size), $\gamma = 0.05$ is a shrinkage factor, $t_0 = 10$ stabilises early iterations, and $\kappa = 0.75$ controls the long-run averaging weight. The running average $\bar{\varepsilon}_m$ is used as the fixed step size after adaptation ends — it is more stable than the instantaneous ε_m . The feedback is straightforward: when $\alpha_m < \delta$ the chain is rejecting too often, so $\bar{H}_m$ increases and ε decreases; when $\alpha_m > \delta$ the opposite happens.

Effective sample size (ESS). Because consecutive MCMC samples are correlated, N correlated samples carry less information than N independent ones. The effective sample size quantifies this:

$$\text{ESS} = \frac{N}{1 + 2 \sum_{k=1}^{\infty} \rho_k}, \quad (130)$$

where $\rho_k = \text{Corr}(\theta_t, \theta_{t+k})$ is the lag- k autocorrelation of the chain. To build intuition: for an AR(1) chain $\theta_t = \phi \theta_{t-1} + \varepsilon_t$ with $|\phi| < 1$, we have $\rho_k = \phi^k$ and $\text{ESS} = N(1 - \phi)/(1 + \phi)$. With $N = 200$ samples, $\phi = 0.9$ gives $\text{ESS} \approx 11$ — the chain is exploring very slowly. $\phi = 0$ gives $\text{ESS} = 200$ — fully independent. In practice the infinite sum is truncated where autocorrelations become negligible; we compute ESS via `tfp.mcmc.effective_sample_size` using FFT-based autocorrelation. With fewer than a few hundred samples the estimator is noisy and ESS can saturate at N even when mixing is imperfect.

$\hat{R}$: Potential Scale Reduction Factor. $\hat{R}$ (Gelman & Rubin, 1992) detects non-convergence by comparing *within-chain* to *between-chain* variance. Since we run a single chain, we split the N post-burn-in samples into two halves of length $n = N/2$ and treat them as two independent chains. Let $\bar{\theta}_j$ and s_j^2 be the mean and variance of chain j ($j = 1, 2$). Define:

$$W = \frac{1}{2}(s_1^2 + s_2^2) \quad (\text{within-chain variance}), \quad (131)$$

$$B = n(\bar{\theta}_1 - \bar{\theta}_2)^2/2 \quad (\text{between-chain variance}), \quad (132)$$

$$\hat{V} = \frac{n-1}{n} W + \frac{1}{n} B \quad (\text{pooled variance estimate}). \quad (133)$$

Then $\hat{R} = \sqrt{\hat{V}/W}$. If the chain has converged, both halves look like draws from the same distribution, so $B \approx 0$ and $\hat{R} \approx 1$. The standard threshold is $\hat{R} > 1.1$ to flag non-convergence. Values slightly below 1 are an artifact of finite-sample estimation and are harmless. With only $n = 100$ samples per half, both W and B are estimated noisily, so at least $N \geq 400$ total samples are recommended before trusting $\hat{R}$.

6.4.2 1-D Model, 1 Parameter

We use a 1-D linear Gaussian model as the minimal end-to-end test of the differentiable pipeline. The model is:

$$\begin{aligned} x_k &= Fx_{k-1} + Bw_k, & w_k &\sim \mathcal{N}(0, 1), \\ z_k &= Hx_k + Dv_k, & v_k &\sim \mathcal{N}(0, 1), \end{aligned} \quad (134)$$

where F, B, H, D are scalar constants. The process noise covariance is $Q = B^2$ and the observation noise covariance is $R = D^2$. We fix F, H , and D at their known values and treat B (equivalently Q) as the single scalar parameter θ to be estimated. The Kalman filter is exact for this model, so we have ground-truth posteriors for both the state estimates and the parameter posterior.

The validation strategy is two-stage. First, we run MAP to confirm that gradients flow correctly through the entire pipeline: filter, log-likelihood, and prior. MAP minimizes $L_{\text{MAP}}(\theta) = -\sum_k \log p_{\theta}(z_k | z_{1:k-1}) - \log p(\theta)$ using Adam with a fresh particle seed at each step. Convergence to the true parameter value indicates the pipeline is numerically sound. Second, we run HMC to recover the full posterior $p(\theta | z_{1:T})$. The posterior should be unimodal and tightly concentrated around the true value, matching the Kalman-based reference. Any discrepancy flags a problem in the gradient computation or the filter itself.

(Numerical results to be filled.)

6.4.3 Multi-Dimensional Model, 1 Parameter

(Results to be filled.)

We increase the state dimension to 3 or 5 while still estimating a single scalar parameter. This tests whether the filter scales correctly with state dimension and whether the gradient computation remains stable. The true posterior is still one-dimensional and analytically tractable via the Kalman filter.

References

- [1] Simo Särkkä. *Bayesian Filtering and Smoothing*. Cambridge University Press, 2013.
- [2] Fred Daum, Jim Huang, and Arjang Noushin. Exact particle flow for nonlinear filters. In *Signal processing, sensor fusion, and target recognition XIX*, volume 7697, pages 92–110. SPIE, 2010.
- [3] Fred Daum and Jim Huang. Particle degeneracy: root cause and solution. In *Signal Processing, Sensor Fusion, and Target Recognition XX*, volume 8050, pages 367–377. SPIE, 2011.
- [4] Yunpeng Li and Mark Coates. Particle filtering with invertible particle flow, 2017.
- [5] Liyi Dai and Fred Daum. A new parameterized family of stochastic particle flow filters, 2021.
- [6] Tao Ding and Mark J. Coates. Implementation of the daum-huang exact-flow particle filter. In *2012 IEEE Statistical Signal Processing Workshop (SSP)*, pages 257–260. IEEE, 2012.
- [7] Manuel Pulido and Peter Jan van Leeuwen. Sequential monte carlo with kernel embedded mappings: The mapping particle filter. 2019.
- [8] Chih-Chi Hu and Peter Jan Van Leeuwen. A particle flow filter for high-dimensional system applications. 2021.
- [9] Sebastian Reich. A nonparametric ensemble transform method for bayesian inference. *SIAM Journal on Scientific Computing*, 35(4):A2013–A2024, 2013.
- [10] Adrien Corenflos, James Thornton, George Deligiannidis, and Arnaud Doucet. Differentiable particle filtering via entropy-regularized optimal transport, 2021.
- [11] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. pages 2292–2300, 2013.
- [12] Shreyas Chaudhari, Srinivasa Pranav, and José M. F. Moura. Gradnetot: Learning optimal transport maps with gradnets. *arXiv preprint arXiv:2507.13191*, 2025. v2.
- [13] Michael Zhu, Kevin P. Murphy, and Rico Jonschkowski. Towards differentiable resampling. *arXiv preprint arXiv:2004.11938*, 2020.
- [14] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam R. Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3744–3753. PMLR, 2019.
- [15] Xiongjie Chen and Yunpeng Li. An overview of differentiable particle filters for data-adaptive sequential bayesian inference. *Foundations of Data Science*, 2023. Also available as arXiv:2302.09639.
- [16] Brandon Amos. Tutorial on amortized optimization: Learning to optimize over continuous spaces. *arXiv preprint arXiv:2202.00665*, 2022. Version 5, October 2025.
- [17] Brandon Amos, Giulia Luise, Samuel Cohen, and Ievgen Redko. Meta optimal transport. In *Proceedings of the 40th International Conference on Machine Learning*, PMLR 202, Honolulu, Hawaii, USA, 2023. arXiv:2206.05262v2.
- [18] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.
- [19] Simon Duane, A.D. Kennedy, Brian J. Pendleton, and Duncan Roweth. Hybrid monte carlo. *Physics Letters B*, 195(2):216–222, 1987.
- [20] Radford M. Neal. Mcmc using hamiltonian dynamics, 2012.
- [21] Matthew D. Hoffman and Andrew Gelman. The no-u-turn sampler: Adaptively setting path lengths in hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15(1):1593–1623, 2014.

- [22] Peter Karkus, David Hsu, and Wee Sun Lee. Particle filter networks with application to visual localization. In *Conference on Robot Learning (CoRL)*, pages 169–178. PMLR, 2018.
- [23] Alexandros Beskos, Natesh Pillai, Gareth O. Roberts, Jesús-Mariá Sanz-Serna, and Andrew M. Stuart. Optimal tuning of the hybrid Monte Carlo algorithm. *Bernoulli*, 19(5A):1501–1534, 2013.